

RHCSA EX200 — Resumo Dia 02/05

Boot Reset · Repositório DNF · Rede (nmcli)

PROVA: 14/05

Pontos Críticos do Dia

Regra	Motivo
ro → rw + init=/bin/bash	Sem isso o <code>passwd</code> falha (shadow precisa ser gravável)
<code>.autorelabel</code>	Obrigatório — sem ele o SELinux bloqueia o login no próximo boot
gpgkey= obrigatório quando <code>gpgcheck=1</code>	Sem a chave, o DNF recusa todos os pacotes
+ antes de <code>ipv4.addresses / ipv6.addresses</code>	Sem o <code>+</code> , o IP secundário sobrescreve o primário

BLOCO 1 — Reset de Senha Root

Procedimento **idêntico** em todos os simulados — só muda a senha. Treine até fazer em menos de 5 minutos, sem consultar anotações.

- Reiniciar → no GRUB, pressionar 'e' no kernel padrão
- Ir até a linha que começa com 'linux'
- Ctrl+E (ir ao final da linha)
- Trocar 'ro' por 'rw'
- Adicionar no final: `init=/bin/bash`
- Ctrl+X para bootar
- No shell root:

passwd root

nova senha pedida no exame

touch /.autorelabel

obrigatório (SELinux)

exec /sbin/init

reinicia o sistema

Exemplo (Simulado 1 — ServerA, senha `password`):

```
passwd root
password
password
touch /.autorelabel
exec /sbin/init
```

Por que cada passo existe:

- ro → rw:** o kernel monta o filesystem read-only por padrão; `passwd` precisa gravar em `/etc/shadow`.
- init=/bin/bash:** inicia um shell direto, pulando o `systemd` — porta de entrada sem senha.
- touch /.autorelabel:** o contexto SELinux do `/etc/shadow` fica incorreto ao ser editado fora do fluxo normal; força o relabel completo no boot seguinte.
- exec /sbin/init:** no shell mínimo, `reboot` pode não existir; substitui o processo atual pelo `init` real.

BLOCO 2 — Repositório DNF (4 variações possíveis)

Estrutura do `.repo` é sempre igual — muda `baseurl` e `gpgcheck`. Pratique as 4.

Variação A — ISO montada com loop (sem fstab, temporário)

```
mount -o loop RHEL-10.iso /mnt
```

```
# /etc/yum.repos.d/local.repo
[BaseOS]
name=RHEL 10 BaseOS
baseurl=file:///mnt/BaseOS
enabled=1
gpgcheck=0

[AppStream]
name=RHEL 10 AppStream
baseurl=file:///mnt/AppStream
enabled=1
gpgcheck=0
```

```
dnf clean all && dnf repolist
```

`file:///` = 3 barras (protocolo + raiz local). `-o loop` cria dispositivo virtual para montar o arquivo ISO.

Variação B — ISO com montagem persistente no fstab

```
mkdir -p /repo
```

```
# /etc/fstab
/RHEL-10.iso /repo iso9660 loop,defaults 0 0
```

```
mount -a
```

```
# /etc/yum.repos.d/local.repo
```

```
[BaseOS]
baseurl=file:///repo/BaseOS
enabled=1
gpgcheck=0

[AppStream]
baseurl=file:///repo/AppStream
enabled=1
gpgcheck=0
```

`iso9660` = filesystem de ISOs/CDs. `0 0` no final = sem dump, sem fsck no boot.

Variação C — Repositório via HTTP (sem montar nada)

```
# /etc/yum.repos.d/http.repo
[BaseOS_HTTP]
name=BaseOS via HTTP
baseurl=http://192.168.1.12/rhel10/BaseOS
enabled=1
gpgcheck=0

[AppStream_HTTP]
baseurl=http://192.168.1.12/rhel10/AppStream
enabled=1
gpgcheck=0
```

```
# se pedir para usar SOMENTE este repo:
dnf config-manager --set-disabled "*"
dnf clean all && dnf repolist
```

Variação D — ISO com GPG check habilitado

```
# /etc/fstab (dispositivo físico)
/dev/sr0 /mnt/repo iso9660 defaults 0 0

# /etc/fstab (arquivo ISO)
/RHEL10.iso /mnt/disc iso9660 defaults,loop 0 0
```

```
mount -a
```

```
# /etc/yum.repos.d/dvd.repo
[BaseOS_DVD]
baseurl=file:///mnt/repo/BaseOS
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release

[AppStream_DVD]
baseurl=file:///mnt/repo/AppStream
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

A chave da Red Hat já vem pré-instalada em `/etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release` — não precisa baixar. Se `gpgcheck=1` e faltar `gpgkey=`, o DNF rejeita todos os pacotes.

BLOCO 3 — Rede (nmcli)

Um único exercício-tipo: criar perfil com IPv4 + IPv6 estático e adicionar IP secundário sem sobrescrever o primário.

Template (decore esta estrutura)

```
# 1. Criar o perfil
nmcli con add con-name NOME ifname enp0s3 type ethernet \
  ipv4.method manual ipv4.addresses IP/MASCARA \
  ipv4.gateway GATEWAY ipv4.dns DNS \
  ipv6.method manual ipv6.addresses IPV6/PREFIXO \
  ipv6.gateway GW6 ipv6.dns DNS6 \
  connection.autoconnect yes

# 2. Adicionar IP secundário (o '+' é obrigatório)
nmcli con mod NOME +ipv4.addresses IP_SECUNDARIO/MASCARA
nmcli con mod NOME +ipv6.addresses IPV6_SECUNDARIO/PREFIXO

# 3. Ativar e verificar
nmcli con up NOME
ip addr show enp0s3
nmcli con show NOME
```

Exemplo completo (Simulado 2 — caso mais completo)

IPv4 192.168.1.3/24 GW .1 DNS 8.8.8.8 | IPv6 fd01::103/64 GW fd01::1 | Sec IPv4 10.0.0.3/24 | Sec IPv6 fd01::200/64

```
nmcli con add con-name enp0s3 ifname enp0s3 type ethernet \
  ipv4.method manual ipv4.addresses 192.168.1.3/24 \
  ipv4.gateway 192.168.1.1 ipv4.dns 8.8.8.8 \
  ipv6.method manual ipv6.addresses fd01::103/64 \
  ipv6.gateway fd01::1 \
  connection.autoconnect yes
```

```
nmcli con mod enp0s3 +ipv4.addresses 10.0.0.3/24
nmcli con mod enp0s3 +ipv6.addresses fd01::200/64

nmcli con up enp0s3
ip addr show enp0s3
```

Parâmetros-chave:

- **ipv4.method manual** → sem isso o NetworkManager usa DHCP.
- **connection.autoconnect yes** → perfil sobe sozinho no boot (sempre usar).
- `ipv6.dns` é parâmetro separado de `ipv6.gateway` — não confundir.
- **+** (mais) → soma ao valor existente; sem ele, sobrescreve o IP primário.
- `con-name` pode ser diferente de `ifname` (ex: `lab-link` para a interface `enp0s3`) — são parâmetros independentes.

✔ Checklist Final

Bloco 1

- ☐ Reset de senha root sem olhar anotações (< 5 min)
- ☐ Sei por que `ro` → `rw`
- ☐ Sei por que `.autorelabel` é obrigatório
- ☐ Sei por que `exec /sbin/init` em vez de `reboot`

Bloco 2

- ☐ `.repo` com ISO via loop (`gpgcheck=0`)
- ☐ `.repo` com ISO persistente no `fstab` (`iso9660` + `loop`)
- ☐ `.repo` via HTTP
- ☐ `.repo` com GPG habilitado + `gpgkey=` correto
- ☐ `dnf config-manager --set-disabled "*"` quando pedido
- ☐ Confirmar com `dnf repolist`

Bloco 3

- ☐ Perfil `nmcli` com IPv4 + IPv6 num único comando
- ☐ IP secundário IPv4 com `+`
- ☐ IP secundário IPv6 com `+`
- ☐ `nmcli con up` + `ip addr show` + `nmcli con show`

RHCSA EX200 — Resumo Dia 03/05

LVM · VDO · Thin Provisioning · Swap · Partição Padrão

PROVA: 14/05

Hierarquia LVM — Memorize

PV — Physical Volume
tijolo (partição)

VG — Volume Group
pilha de tijolos

LV — Logical Volume
estrutura construída

⚠ Armadilhas Críticas

Certo	Errado
<code>-L</code> (maíúsculo) = tamanho em MB/GB	<code>-l</code> (minúsculo) = número de extents
<code>lvextend -r</code> = expande LV + FS	sem <code>-r</code> = só o LV; o FS não cresce
Swap no fstab: <code>none</code> como mountpoint	usar <code>/swap</code> como mountpoint
Partição padrão: <code>mkfs</code> direto	usar <code>pvccreate</code> em partição padrão

BLOCO 1 — LVM Padrão

Fluxo completo (decore a sequência):

`fdisk → pvccreate → vgcreate → lvcreate → mkfs → fstab → lvextend -r`

```
# 1. Criar partição
fdisk /dev/sdb
    n → p (ou Enter em GPT) → Enter → Enter → +2G → t → 8e (Linux LVM) → w
partprobe /dev/sdb          # obrigatório

# 2. Physical Volume
pvccreate /dev/sdb1

# 3. Volume Group (PE padrão 4MB)
vgcreate myvg /dev/sdb1

# 4. Logical Volume
lvcreate -n mylv -L 500M myvg      # -L = tamanho
# lvcreate -n mylv -l 50 myvg      # -l = n° de extents

# 5. Formatar
mkfs.ext4 /dev/myvg/mylv

# 6. Montar persistente
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a
df -h /mylv

# 7. Expandir LV + filesystem em UM comando
lvextend -r -L +500M /dev/myvg/mylv

# Verificação
pvs && vgs && lvs && lsblk
```

Exemplo completo (S1-Q6): disco `/dev/sdb`, VG `myvg`, LV `mylv` 500MiB → ext4 → `/mylv`, expandir +500MiB

```
fdisk /dev/sdb          # n → p → Enter → Enter → +1G → t → 8e → w
partprobe /dev/sdb
pvccreate /dev/sdb1
vgcreate myvg /dev/sdb1
lvcreate -n mylv -L 500M myvg
mkfs.ext4 /dev/myvg/mylv
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a && df -h /mylv
lvextend -r -L +500M /dev/myvg/mylv
df -h /mylv
```

Crie a partição com `+1G` para ter espaço para os 500M iniciais + a expansão de +500M. `lvextend` sem `-r` expande apenas o container LV — o filesystem **não** cresce. `partprobe` é obrigatório após `fdisk`.

BLOCO 2 — PE Size Personalizado

O que é PE (Physical Extent)? É o bloco mínimo de alocação do LVM — como a unidade de cluster de um filesystem. Padrão: 4MB. Quando o exame pede PE de 16M, use `-s 16M` no `vgcreate`. LVs desse VG só podem ser criados em múltiplos de 16MB.

A conta que cai na prova:

- `50 extents × 16MB = 800MB` (tamanho resultante do LV)
- `800MB ÷ 16MB = 50 extents` (conta inversa)

Exemplo completo (S5-Q15): VG `myvg` com PE size 16MiB, LV `mylv` com 50 extents, xfs → `/mnt/mylv`

```
fdisk /dev/sdb      # n → Enter → Enter → +2G → t → 8e → w
partprobe /dev/sdb
pvcreate /dev/sdb1
vgcreate -s 16M myvg /dev/sdb1      # PE size personalizado
lvcreate -l 50 -n mylv myvg         # -l minúsculo = extents (50 × 16MB = 800MB)
mkfs.xfs /dev/myvg/mylv
mkdir /mnt/mylv
echo "/dev/myvg/mylv /mnt/mylv xfs defaults 0 0" >> /etc/fstab
mount -a
vgdisplay myvg | grep "PE Size"      # deve mostrar 16.00 MiB
lvdisplay /dev/myvg/mylv | grep "Current LE"  # deve mostrar 50
```

-l minúsculo = número de extents | -L maiúsculo = tamanho em MB/GB — não confundir!

BLOCO 3 — Partição Padrão (sem LVM)

Quando NÃO usar LVM: quando a questão pede "partição padrão", "formatar diretamente", ou não menciona PV/VG/LV. Fluxo: `fdisk → mkfs` (pula tudo do meio).
Por que UUID no fstab? Nomes de dispositivo (`/dev/sdb1`) podem mudar entre reboots. O UUID é permanente — identifica a partição pelo conteúdo, não pelo nome do dispositivo. Use `blkid` para obtê-lo.

Exemplo completo (S3-Q6): partição 512MiB em `/dev/sdb → ext4 → /mnt/data` via UUID, persistente

```
fdisk /dev/sdb      # n → Enter → Enter → +512M → w
partprobe /dev/sdb
mkfs.ext4 /dev/sdb1  # NÃO usar pvcreate aqui!
mkdir -p /mnt/data
blkid /dev/sdb1      # copiar o UUID mostrado
vim /etc/fstab
# UUID=<uuid-aqui> /mnt/data ext4 defaults 0 0
mount -a
df -h /mnt/data
```

Esta questão pede partição **padrão**. NÃO executar `pvcreate / vgcreate / lvcreate`.

BLOCO 4 — Swap

Três diferenças em relação a partições comuns:

- Tipo no fdisk: **82** (Linux Swap), não 8e (Linux LVM)
- Formatador: `mkswap`, não `mkfs`
- fstab: ponto de montagem = **none** | tipo = **swap**

Exemplo completo (S1-Q15): partição swap 512MiB

```
fdisk /dev/sdb
  n → Enter → Enter → +512M → t → 82 → w  # 82 = Linux Swap
partprobe /dev/sdb
mkswap /dev/sdb2      # ajustar número da partição
blkid /dev/sdb2       # copiar UUID
vim /etc/fstab
# UUID=<uuid> none swap defaults 0 0
#      ^^^^^ SEMPRE 'none' — nunca /swap ou outro caminho
swapon -a             # ativar sem reboot
free -h               # confirmar coluna Swap aumentou
```

Simulado	Tamanho	Partição típica
S1	512MiB	/dev/sdb2
S2	500MiB	/dev/sdb2
S5	1GiB	/dev/sdb2
S6	1GiB	/dev/sdb2

`swapon -a` ativa tudo do fstab com tipo swap — bom teste de que o fstab está correto. O número da partição depende de quantas já existem no disco.

BLOCO 5 — VDO & Thin Provisioning

VDO (Virtual Data Optimizer)

Tecnologia de deduplicação e compressão inline gerenciada pelo LVM.

- `--size` = espaço físico real consumido no VG
- `--virtualsize` = espaço apresentado ao SO (pode ser 4x-10x maior)
- RHEL 10 exige **mínimo ~5GB físico** para VDO (metadados do índice UDS)

Exemplo completo (S1-Q20): VG `myvg` já existente, VDO LV `vdo_lv`, 5GiB físico → 20GiB virtual, xfs → `/vdo_data`

```
dnf install lvm2 vdo -y
lvcreate --type vdo --name vdo_lv --size 5G --virtualsize 20G myvg
mkfs.xfs /dev/myvg/vdo_lv
mkdir /vdo_data
echo "/dev/myvg/vdo_lv /vdo_data xfs defaults 0 0" >> /etc/fstab
mount -a
vdostats --human-readable
lsblk
```

Thin Provisioning

Permite prometer mais espaço do que existe fisicamente. Dois objetos: **Thin Pool** (container físico, `-T` + `-L`) e **Thin Volume** (virtual, `-v` + `-T` referenciando o pool).

Exemplo completo (S3-Q27): VG `thin_vg`, Thin Pool `my_thin_pool` 500MiB, Thin Volume `thin_vol` 2GiB virtual, xfs → `/mnt/thin`

```
fdisk /dev/sdb          # criar partição 3, +1G, tipo 8e
partprobe /dev/sdb
pvcreate /dev/sdb3
vgcreate thin_vg /dev/sdb3

lvcreate -L 500M -T thin_vg/my_thin_pool          # cria o pool (500MB físico real)
lvcreate -V 2G -T thin_vg/my_thin_pool -n thin_vol # cria o volume (2GB virtual > pool!)

mkfs.xfs /dev/thin_vg/thin_vol
mkdir /mnt/thin
echo "/dev/thin_vg/thin_vol /mnt/thin xfs defaults 0 0" >> /etc/fstab
mount -a
lvs thin_vg      # mostra VSize (2G) e o pool (500M)
```

`-T` cria/referencia o thin pool. `-v` define o tamanho virtual. Sem `-T` na criação do pool: não é thin. Sem `-v` no volume: não é thin.

Técnica extra — `lvrename` (pode cair combinado com expansão do pool):

```
lvextend -L +500M vg1/thinpool1          # expandir o pool
lvrename vg1/thinpool1 vg1/thinpool2     # renomear pool
lvrename vg1/thinvol1 vg1/thinvol2       # renomear volume
```

Sintaxe: `lvrename VG/LV_ANTIGO VG/LV_NOVO` — o VG é obrigatório. **Não use** `mv` no `/dev` — isso só renomeia o symlink, não os metadados LVM internos.

📋 Tabela de Diferenças Críticas

Situação	Comando CORRETO	Erro comum
LV por tamanho	<code>lvcreate -L 500M myvg</code>	<code>lvcreate -l 500M</code>
LV por extents	<code>lvcreate -l 50 myvg</code>	<code>lvcreate -L 50</code>
Expandir + FS	<code>lvextend -r -L +500M ...</code>	esquecer o <code>-r</code>
PE personalizado	<code>vgcreate -s 16M vg /dev/sdb1</code>	criar VG sem <code>-s</code>
Swap no fstab	<code>UUID=xx none swap defaults 0 0</code>	<code>UUID=xx /swap swap ...</code>
Partição padrão	<code>mkfs.ext4 /dev/sdb1</code>	<code>pvcreate /dev/sdb1</code> (cria LVM)
VDO mínimo	<code>--size 5G</code>	<code>--size 1G</code> (falha por metadados)
Thin pool	<code>lvcreate -L 500M -T vg/pool</code>	esquecer o <code>-T</code>
Thin volume	<code>lvcreate -V 2G -T vg/pool -n vol</code>	esquecer o <code>-v</code>

✅ Checklist Final

LVM Padrão

- ☐ Fluxo completo (fdisk → pvs → vgs → lvs) sem consulta
- ☐ `lvextend -r` funciona — filesystem cresce junto com o LV
- ☐ fstab correto para LVM (`/dev/vg/lv`) + `mount -a` sem erro

PE Size Personalizado

- ☐ `vgcreate -s 16M` para PE personalizado
- ☐ `-l` (extents) vs `-L` (tamanho) — não confundir
- ☐ Sei fazer a conta: 50 extents × 16MB = 800MB

Partição Padrão

- ☐ `mkfs` direto na partição, SEM `pvcreate/vgcreate/lvcreate`
- ☐ UUID no fstab — obtido com `blkid`

Swap

- ☐ `fdisk` tipo 82 (swap, não 8e de LVM)
- ☐ fstab com `none` como mountpoint e `swap` como tipo
- ☐ `swapon -a` ativa | `free -h` confirma

VDO & Thin Provisioning

- ☐ VDO: `--size` (físico) vs `--virtualsize` (apresentado ao SO)
- ☐ VDO mínimo 5GB físico
- ☐ Thin pool: `lvcreate -L -T` | Thin volume: `lvcreate -V -T`
- ☐ `lvrename vg/old vg/new` funciona em pools e volumes

RHCSA EX200 — Resumo Dia 04/05

Usuários · Grupos · Permissões · ACL · Umask · Sudo

PROVA: 14/05

⚠️ Armadilhas do Dia

Certo	Errado
<code>usermod -aG grupo user</code>	sem <code>-a</code> remove TODOS os outros grupos
<code>chmod 2770</code> (com o 2)	sem o 2 não há SGID
<code>setfacl -m d:grupo:rx</code>	sem <code>d:</code> novos arquivos ignoram a ACL
umask no <code>.bashrc</code>	umask no terminal some ao fechar a sessão
<code>NOPASSWD: /usr/bin/dnf</code>	caminho relativo <code>dnf</code> é inseguro

📊 Tabela chmod Especial

Dígito	Nome	Efeito
4xxx	SUID	Executável roda com permissão do dono
2xxx	SGID	Diretório: novos arquivos herdam o grupo
1xxx	Sticky	Só o dono do arquivo pode deletá-lo

BLOCO 1 — Usuários e Grupos

Flags essenciais do `useradd` :

- `-u` `UID` — UID fixo. Sem isso o sistema escolhe o próximo disponível.
- `-s` `SHELL` — Shell do usuário. `/sbin/nologin` rejeita conexões interativas — contas de serviço.
- `-e` `DATA` — Data de expiração. Formato obrigatório: `YYYY-MM-DD` .
- `-G` `GRUPOS` — Grupos secundários. Vírgula separa múltiplos: `-G g1,g2` .

```
useradd alex                                # usuário simples
useradd -u 1500 -s /sbin/nologin sam        # UID fixo + conta de serviço
useradd -u 1250 -e 2029-12-21 john         # UID + expiração
useradd -G developers alex                 # grupo secundário
useradd -G accounting,finance alex         # múltiplos grupos

usermod -s /bin/bash sam                   # mudar shell
usermod -aG developers alex                # ADICIONAR grupo (mantém outros)
usermod -u 2000 alex                       # mudar UID

groupadd developers

# Verificação
id alex
grep alex /etc/passwd
chage -l john
su - harry      # testar nologin
```

Exemplo completo (S3-Q7): mudar shell de `sam` para bash + criar `john` com UID 1250 e expiração

```
usermod -s /bin/bash sam
useradd -u 1250 -e 2029-12-21 john
```

`usermod -G` (sem `-a`) **substitui** todos os grupos secundários. Sempre use `-aG` para adicionar sem remover os existentes. Formato de data obrigatório: `YYYY-MM-DD` .

BLOCO 2 — Políticas Globais

Quatro arquivos, quatro funções completamente diferentes:

/etc/skel — Template do home

Tudo aqui é copiado automaticamente para o home de **novos** usuários. Não afeta usuários já existentes.

Exemplo (S1-Q12):

```
touch /etc/skel/Welcome.txt
useradd testuser123
ls /home/testuser123/      # o arquivo deve aparecer aqui
```

/etc/login.defs — Expiração de senha

`PASS_MAX_DAYS` define o máximo de dias que uma senha pode ser usada. Afeta apenas usuários criados **depois** da alteração. Para usuários existentes: `chage -M 90 username` .

Exemplo (S1-Q12):

```
vim /etc/login.defs
# PASS_MAX_DAYS 90
```

/etc/security/pwquality.conf — Complexidade de senha

`minlen` define o número mínimo de caracteres.

Exemplo (S1-Q12):

```
vim /etc/security/pwquality.conf
# minlen = 8
```

/etc/profile.d/ — Variáveis globais

Qualquer arquivo `.sh` aqui roda no login de **todos** os usuários — mais seguro que editar `/etc/profile` direto.

Exemplo completo (S6-Q28):

```
vim /etc/profile.d/custom_env.sh
# export VAR="RHCSA"
# export HISTFILESIZE=2000
source /etc/profile.d/custom_env.sh
echo $VAR           # RHCSA
echo $HISTFILESIZE   # 2000
```

BLOCO 3 — Permissões Especiais

SGID em diretório (chmod 2xxx)

Arquivos criados dentro herdam o **grupo do diretório**, não o grupo primário do criador. Essencial para colaboração em equipe.

Exemplo completo (S1-Q13): grupo `developers`, `/opt/dev-data` com SGID, owner+group=RWX, others sem acesso

```
groupadd developers
mkdir /opt/dev-data
chown :developers /opt/dev-data      # grupo do diretório
chmod 2770 /opt/dev-data             # 2=SGID | 7=owner rwx | 7=group rwx | 0=others
ls -ld /opt/dev-data
# drwxrws--- (o 's' no lugar do 'x' do grupo = SGID ativo)
```

`chmod 2770` vs `chmod 770` : sem o `2` não há SGID e arquivos novos herdam o grupo primário do criador, não o grupo do diretório. Sempre `chown :grupo` antes do `chmod 2xxx` .

Sticky Bit (chmod 1xxx)

Cada usuário só pode deletar os próprios arquivos no diretório (resolve o problema do `/tmp`).

Exemplo completo (S3-Q23): `/data/shared` , permissão 777, sticky bit

```
mkdir -p /data/shared
chmod 1777 /data/shared             # 1=sticky | 777=todos rwx
ls -ld /data/shared
# drwxrwxrwt (o 't' no lugar do 'x' de others = sticky ativo)
```

Combinação avançada — SGID + ACL cross-group (S5-Q14/S6-Q17):

```
groupadd Innovation && groupadd Solution
mkdir -p /groups/Innovation /groups/Solution
chown :Innovation /groups/Innovation
chown :Solution /groups/Solution
chmod 2770 /groups/Innovation /groups/Solution

setfacl -m g:Solution:rx /groups/Innovation      # acesso atual
setfacl -m d:g:Solution:rx /groups/Innovation    # herança futura
getfacl /groups/Innovation
```

São **dois** `setfacl` : sem `d:` (diretório agora) e com `d:` (arquivos futuros). Esquecer o `d:` é o erro mais comum.

BLOCO 4 — ACL (Access Control Lists)

Quando usar **ACL vs chmod**: `chmod` trabalha só com owner/group/others. ACL permite permissões para usuários/grupos específicos além dessas três.

```
# setfacl -m TIPO:NOME:PERMS arquivo | TIPO: u (user) | g (group) | d (default/herança)
setfacl -m u:alex:rw /arquivo        # alex: leitura + escrita
setfacl -m u:kevin:--- /arquivo      # kevin: SEM acesso
setfacl -m g:finance:rx /diretorio   # grupo: leitura + execução
setfacl -m d:g:finance:rx /diretorio # DEFAULT ACL — herança para novos arquivos

getfacl /arquivo                     # lista todas as ACLs
ls -l /arquivo                       # '+' no final = tem ACL

setfacl -x u:alex /arquivo           # remover ACL do alex
setfacl -b /arquivo                  # remover TODAS as ACLs
```

Exemplo completo (S2-Q15): `/etc/fstab` → `/var/tmp/fstab` , owner+group=root, remover execute de todos, `stewart` =RW, `kevin` =sem acesso

```
cp /etc/fstab /var/tmp/fstab
chown root:root /var/tmp/fstab
chmod a-x /var/tmp/fstab             # remove execute de owner+group+others
setfacl -m u:stewart:rw /var/tmp/fstab
setfacl -m u:kevin:--- /var/tmp/fstab
getfacl /var/tmp/fstab
```

O `+` no `ls -l` indica ACL ativa. A `mask` no `getfacl` é o teto máximo para qualquer ACL (exceto owner).

BLOCO 5 — Umask

Umask define as permissões **removidas** dos valores base: arquivo base=666 (sem execute), diretório base=777.

Umask	Arquivo (cálculo)	Arquivo	Dir	Uso típico
022	666-022=644	rw-r--r--	755 rwxr-xr-x	Padrão do sistema
027	666-027=640	rw-r-----	750 rwxr-x---	Segurança moderada
077	666-077=600	rw-----	700 rwx-----	Máxima privacidade
002	666-002=664	rw-rw-r--	775 rwxrwxr-x	Colaboração em grupo

Exemplo completo (S1-Q28): umask 027 para `harry`, persistente (640/750)

```
vim /home/harry/.bashrc
# umask 027

su - harry
umask                # 0027
touch testfile && ls -l testfile  # -rw-r----- (640)
mkdir testdir && ls -ld testdir   # drwxr-x--- (750)
```

`.bashrc` roda em todo shell interativo. `.bash_profile` só no login inicial. Umask no terminal = temporário; no `.bashrc` = permanente.

BLOCO 6 — Sudo (sudoers.d) + Links Hard e Soft

Sudo com /etc/sudoers.d/

Arquivos aqui são incluídos automaticamente no sudo. Um erro de sintaxe não quebra o sudo inteiro — mais seguro que editar `/etc/sudoers` direto.

Exemplo completo (S1-Q19):

```
useradd alex
vim /etc/sudoers.d/alex
# alex ALL=(ALL) NOPASSWD: /usr/bin/dnf
#                               ^^^^^^^^^^^^^^ CAMINHO ABSOLUTO obrigatório

which dnf                # confirma o caminho: /usr/bin/dnf

su - alex
sudo dnf repolist        # deve funcionar SEM senha
sudo reboot              # deve ser negado ou pedir senha
```

Sempre caminho absoluto. Caminho relativo (`dnf`) é inseguro — manipulação do PATH poderia enganar o comando.

Links Hard e Soft

Hard link: dois nomes apontando para o **mesmo inode**. Soft link: um ponteiro para o **caminho** de outro arquivo.

Exemplo completo (S2-Q24):

```
touch /home/root/data.txt

ln /home/root/data.txt /home/root/data-hard      # HARD LINK (sem -s)
ln -s /home/root/data.txt /var/tmp/data-soft     # SOFT LINK (com -s)

ls -li /home/root/data*                          # hard links têm o MESMO número de inode
ls -l /var/tmp/data-soft                        # data-soft -> /home/root/data.txt
```

	Hard Link	Soft Link
Flag	<code>ln</code> (sem flag)	<code>ln -s</code>
O que armazena	Mesmo inode	Caminho do alvo
Se original é deletado	Link continua funcionando	Link quebra
Entre filesystems	Não funciona	Funciona
Para diretórios	Não permitido	Permitido

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
Adicionar grupo	<code>usermod -aG grupo user</code>	<code>usermod -G grupo user</code> (remove outros!)
SGID em diretório	<code>chmod 2770 dir</code> + <code>chown :grupo</code>	Esquecer o <code>chown :grupo</code>
ACL negar acesso	<code>setfacl -m u:user:--- arq</code>	Não criar a ACL (others acessa)
Default ACL	<code>setfacl -m d:g:grupo:rx dir</code>	Só <code>setfacl</code> sem <code>d:</code> (novos ignoram)
Umask persistente	<code>echo 'umask 027' >> ~/.bashrc</code>	<code>umask 027</code> no terminal (só a sessão)
Sudo sem senha	<code>NOPASSWD: /usr/bin/dnf</code>	<code>NOPASSWD: dnf</code> (caminho relativo)

Checklist Final

Usuários e Grupos

- `useradd` com `-u` (UID), `-s` (shell), `-e` (expiração), `-G` (grupos)
- `usermod -aG` para adicionar grupo SEM remover os outros
- Verificar com `id`, `grep /etc/passwd`, `chage -l`

Políticas Globais

- `touch /etc/skel/arquivo` → aparece no home de novos usuários
- `/etc/login.defs` → `PASS_MAX_DAYS` para expiração de senha
- `/etc/security/pwquality.conf` → `minlen` para tamanho mínimo
- `/etc/profile.d/custom.sh` → variáveis globais para todos

Permissões Especiais

- `chmod 2770` = SGID + owner rwx + group rwx + others ---
- `chmod 1777` = Sticky + todos rwx
- `ls -ld` mostra `s` no group = SGID ativo, `t` em others = Sticky ativo
- SEMPRE `chown :grupo` antes do `chmod 2xxx`

ACL

- `setfacl -m u:user:rw arquivo` (permissão por usuário)
- `setfacl -m u:user:--- arquivo` (negar acesso via ACL)
- `setfacl -m d:g:grupo:rx dir` (Default ACL = herança)
- `getfacl` confirma ACLs; `ls -l` mostra `+` quando há ACL

Umask

- `umask 027` → arquivo 640 | diretório 750
- `umask 077` → arquivo 600 | diretório 700 (máxima privacidade)
- umask no `.bashrc` para persistência por usuário

Sudo e Links

- `/etc/sudoers.d/user` com caminho ABSOLUTO do comando
- `NOPASSWD: /usr/bin/dnf` (não `NOPASSWD: dnf`)
- `ln` = hard link (mesmo inode) | `ln -s` = soft link (ponteiro para caminho)

RHCSA EX200 — Resumo Complemento

Ferramentas Essenciais: vim · man/info · awk · pipes · find avançado · compressão · gerenciamento de arquivos

DOMÍNIO 1 — ESSENTIAL TOOLS

! Armadilhas do Dia

Certo	Errado
vim é a ferramenta obrigatória no exame (sem nano)	Assumir que o nano vai estar disponível no ambiente
Travou no vim? Esc várias vezes + :q! para sair sem salvar	Fechar o terminal — o exame percebe sessões abertas
man COMANDO → /EXAMPLES → n resolve a maioria dos bloqueios	Perder tempo tentando lembrar o comando de cabeça
gzip/bzip2/xz -k MANTÉM o arquivo original	Esquecer a flag -k e perder o arquivo original
ln = hard link (mesmo inode, sobrevive à exclusão do original)	Confundir com ln -s (quebra se o original for deletado)
find ... -exec cmd {} + processa todos de uma vez (mais rápido)	Só conhecer \; — mais lento com muitos arquivos
which confirma o caminho ABSOLUTO do binário (essencial em sudoers)	Usar caminho relativo — inseguro e pode falhar

Tabela-Chave — Mapa de Blocos

Bloco	Conteúdo	Por que importa
1	VIM — editor obrigatório na prova	Ambiente do exame não tem nano
2	man · info · --help · /usr/share/doc	Documentação disponível durante a prova
3	awk — processamento de texto estruturado	Extraír campos de /etc/passwd e similares
4	Pipes avançados, tee e here-doc	I/O redirect é cobrado diretamente
5	find — padrões avançados: -o, !, -perm, -newer	Completa o que falta do find básico
6	which · whereis · type · locate	Localizar binários e arquivos no sistema
7	Gerenciamento de arquivos — cp -a, ln, stat	Hard link vs soft link cai na prova
8	Compressão standalone — gzip, bzip2, xz, zip	Sem tar — comprimir arquivo individual

BLOCO 1 — VIM: Editor Obrigatório na Prova

O exame NÃO permite nano por padrão no ambiente RHEL 10 — vim é a ferramenta esperada. O vim tem 3 modos: Normal (navegação e comandos — modo inicial), Insert (digitar texto — entrar com i, a, o) e Command (salvar, sair, substituir — entrar com :). Se travar, pressione Esc várias vezes até voltar ao modo Normal.

Salvar e sair — os mais cobrados na prova:

Comando	Ação
:w	Salvar SEM sair
:q	Sair (falha se houver mudanças não salvas)
:wq	Salvar E sair (mais usado)
:q!	Sair SEM salvar (forçado) — descartar mudanças
:x	Salvar e sair (igual a :wq)
ZZ	Atalho no modo Normal para :wq

Exemplo típico do exame: substituição global

```
# Buscar (modo Normal)
/palavra          # buscar para frente (n = próxima, N = anterior)

# SUBSTITUIÇÃO GLOBAL – mais cobrada na prova
:%s/antigo/novo/g  # substituir TODAS as ocorrências no arquivo inteiro
:s/antigo/novo/g   # substituir apenas na linha atual
:%s/antigo/novo/gc # perguntar em cada substituição (c = confirm)

# Tarefa: 'Substituir DEVICE por INTERFACE em /etc/sysconfig/network'
vim /etc/sysconfig/network
:%s/DEVICE/INTERFACE/g
:wq
```

Navegação, copiar/colar e desfazer:

Tecla	Ação
gg / G	Primeira linha / última linha do arquivo
:42	Ir para a linha 42
O / \$	Início / fim da linha atual
yy / 5yy	Copiar linha atual / copiar 5 linhas
dd / 5dd	Cortar linha atual / cortar 5 linhas
p / P	Colar depois / antes do cursor
u / Ctrl+R	Desfazer / refazer
:set nu	Exibir números de linha

■ Se travar no vim: pressione Esc várias vezes para garantir que está no modo Normal, depois use :q! para sair sem salvar. Nunca feche o terminal — o exame percebe sessões abertas.

BLOCO 2 — man · info · --help · /usr/share/doc

O EX200 permite o uso do man durante a prova. Saber buscar rapidamente DENTRO do man é a diferença entre lembrar o comando ou encontrá-lo em 20 segundos. Treine o fluxo: man COMANDO → /EXAMPLES → n → q.

```
# ABRIR o manual de um comando
man comando          # abre o manual (q para sair)
man 5 passwd         # seção 5 = formatos de arquivo
man 8 fdisk          # seção 8 = comandos de administração

# BUSCAR em todos os man pages
man -k selinux       # = apropos selinux
apropos firewall     # buscar por palavra-chave

# DENTRO DO MAN – navegação
/palavra             # buscar dentro do manual
n                    # próxima ocorrência
N                    # ocorrência anterior
q                    # sair do manual
```

Man pages mais úteis no EX200:

Man page	O que buscar dentro	Por que usar
man semanage-port	/EXAMPLES	Comando exato para adicionar porta ao SELinux
man nmcli-examples	/ipv4	Exemplos completos de configuração de rede
man 5 crontab	/EXAMPLE	Exemplos de entradas crontab
man setsebool	/EXAMPLE	Sintaxe do setsebool com -P
man semanage-fcontext	SEE ALSO / EXAMPLES	fcontext -a -t e restorecon
man firewall-cmd	/--add-port	Sintaxe exata do firewall-cmd

Man page	O que buscar dentro	Por que usar
man lvmextend	/EXAMPLE	Sintaxe do lvmextend -r
man tar	/EXAMPLE	Flags de compressão e extração

--HELP – *resumo rápido das opções*
comando --help

INFO – *documentação GNU completa (mais detalhada que man)*
info tar
info coreutils

/usr/share/doc – *documentação instalada com os pacotes*
ls /usr/share/doc/ # listar pacotes com documentação
ls /usr/share/doc/httpd/ # docs do Apache

■ Fluxo de uso na prova: 1) tentar lembrar o comando. 2) Se não lembrar: man COMANDO → /EXAMPLES → Enter. 3) Se não souber o nome: apropos PALAVRA-CHAVE. Isso resolve 95% dos bloqueios.

BLOCO 3 — awk: Processamento de Texto Estruturado

O awk divide cada linha em campos por delimitador (padrão = espaço/tab). \$1 = primeiro campo, \$2 = segundo, ..., \$NF = último campo. NR = número da linha atual. Para delimitador diferente (como ':'), use -F:.

Linha de /etc/passwd: root:x:0:0:root:/root:/bin/bash — com -F: os campos são \$1=root, \$2=x, \$3=0, \$4=0, \$5=root, \$6=/root, \$7=/bin/bash.

```
# Imprimir campo específico
awk -F: '{print $1}' /etc/passwd # delimitador ':' → username
awk -F: '{print $1, $3}' /etc/passwd # username e UID

# Filtrar por condição
awk -F: '$3 >= 1000 {print $1}' /etc/passwd # UIDs de usuários normais
awk -F: '$7 == "/bin/bash" {print $1}' /etc/passwd # usuários com bash

# Combinado com outros comandos via pipe
df -h | awk '{print $1, $5}' # filesystem e % de uso
ps aux | awk '$3 > 5.0 {print $1,$3}' # processos com CPU > 5%

# Variáveis especiais
awk '{print NF}' arquivo # NF = número de campos na linha
awk '{print NR, $0}' arquivo # NR = número da linha + linha inteira
```

cut — alternativa mais simples para campos fixos:

```
cut -d: -f1 /etc/passwd # campo 1 com delimitador ':' = username
cut -d: -f1,3 /etc/passwd # campos 1 E 3 = username + UID
cut -d: -f1-3 /etc/passwd # campos 1 até 3
cut -c1-5 arquivo # primeiros 5 caracteres de cada linha
```

■ No EX200, awk geralmente não é exigido explicitamente. Se a questão pede 'extrair campo X', cut -d: -fN é mais simples de lembrar. Use awk quando precisar FILTRAR por condição (ex: UID >= 1000).

BLOCO 4 — Pipes Avançados · tee · here-doc

O objetivo 'Use input-output redirection' cobre pipes, here-docs e manipulação de stdin/stdout/stderr. Pipes encadeiam comandos: a saída de um vira a entrada do próximo.

```
cat /etc/passwd | grep -v '#' | sort # filtrar e ordenar
df -h | grep -v tmpfs | sort -k5 -rn # disco por uso desc.
ps aux | sort -k3 -rn | head -10 # top 10 por CPU
cat /etc/passwd | cut -d: -f1 | sort | uniq # usernames únicos ordenados
```

tee — saída para arquivo E tela simultaneamente:

```
comando | tee arquivo.txt          # sobrescreve o arquivo
comando | tee -a arquivo.txt       # acrescenta ao arquivo (append)
```

Exemplo prático:

```
dnf install httpd -y | tee /tmp/install.log
```

here-doc — entrada multi-linha sem arquivo intermediário:

```
# Criar arquivo de repositório DNF
cat > /etc/yum.repos.d/local.repo << 'EOF'
[BaseOS]
name=BaseOS
baseurl=file:///mnt/BaseOS
enabled=1
gpgcheck=0
EOF
```

Adicionar múltiplas linhas ao /etc/hosts

```
cat >> /etc/hosts << 'EOF'
192.168.1.10 servidor1
192.168.1.11 servidor2
EOF
```

■ As aspas simples em 'EOF' previnem a expansão de variáveis dentro do bloco — importante ao criar arquivos de configuração.

Redirecionamento — tabela de referência:

Operador	Ação	Exemplo
>	Redirecionar stdout (sobrescreve)	cmd > arquivo.txt
>>	Redirecionar stdout (acrescenta)	cmd >> arquivo.txt
<	Redirecionar stdin (ler de arquivo)	cmd < entrada.txt
2>	Redirecionar stderr (sobrescreve)	cmd 2> erros.txt
&>	stdout + stderr juntos	cmd &> tudo.txt
2>&1	stderr para onde stdout aponta	cmd > arq.txt 2>&1
2>/dev/null	Suprimir erros	find / -name x 2>/dev/null
tee	Para tela E arquivo ao mesmo tempo	cmd tee arq.txt
<< 'EOF'	Here-doc (entrada multi-linha)	cat > arq << 'EOF' ... EOF

BLOCO 5 — find: Padrões Avançados

Este bloco completa os filtros básicos de find com operadores lógicos (-o, !), ações (-delete, -exec +), filtros de permissão SUID/SGID e comparação por data.

Operadores lógicos — AND, OR e NOT:

```
# AND (padrão — implícito entre filtros)
find /etc -type f -size +5M          # AND implícito

# OR — qualquer um dos filtros
find /etc -name '*.conf' -o -name '*.cfg'

# NOT — negação
find /etc ! -name '*.conf'          # tudo EXCETO .conf
find / -type f ! -user root 2>/dev/null # não pertence ao root

# Combinando AND + OR com parênteses (escape)
find /home \( -name '*.sh' -o -name '*.py' \) -type f
```

Ações -delete, -exec: \; vs +

```
# -delete – remover diretamente (cuidado!)
find /tmp -type f -empty -delete

# -exec com \; – executa UMA VEZ por arquivo encontrado
find /etc -name '*.conf' -exec chmod 644 {} \;

# -exec com + – passa TODOS os arquivos de uma vez (mais rápido)
find /etc -size +1M -exec cp {} /backup/ +
```

Filtros de permissão — SUID, SGID:

```
find / -perm -4000 2>/dev/null # SUID ativo (numérica)
find / -perm /u=s 2>/dev/null # SUID (simbólica)
find / -perm -2000 2>/dev/null # SGID ativo
find / -perm -6000 2>/dev/null # SUID + SGID juntos
```

Filtros de data e hora:

```
find /var/log -mtime -1 # modificado nas últimas 24h
find /home -atime +30 # ACESSADO há mais de 30 dias
find /etc -newer /etc/passwd # modificados DEPOIS de /etc/passwd
```

BLOCO 6 — Localizar Arquivos: which · whereis · type · locate

Comandos para localizar binários, man pages e arquivos no sistema. which é especialmente útil para confirmar o caminho absoluto de um binário (necessário em sudoers e scripts).

Comando	O que retorna	Exemplo
which dnf	Caminho do binário no PATH atual	/usr/bin/dnf
whereis passwd	Binário + man page + código fonte	passwd: /usr/bin/passwd /usr/share/man/...
type ls	Como o shell interpreta o comando	ls é alias de 'ls --color=auto'
locate httpd.conf	Busca rápida no banco de dados	resultado instantâneo (pode estar desatualizado)
updatedb	Atualizar banco do locate	rodar como root antes de usar locate
locate -i nome	Busca case-insensitive	locate -i HTTPD = httpd ou HTTPD

■ No exame: use find. locate pode não existir no ambiente ou estar com banco desatualizado. which é o mais útil dos quatro — which bash retorna /usr/bin/bash para usar em shebangs.

BLOCO 7 — Gerenciamento de Arquivos: cp -a · ln · stat

Operações de arquivo parecem triviais, mas têm opções específicas que o exame testa. Especialmente: cp -a (copiar tudo incluindo SELinux), ln vs ln -s (hard vs soft link) e stat para ver permissões numéricas e inode.

cp — flags importantes:

Flag	Ação	Quando usar
-r	Recursivo (necessário para diretórios)	cp -r dir/ destino/
-a	Archive: tudo (perms + dono + links + SELinux)	Copiar preservando contexto SELinux
-p	Preservar permissões e timestamps	Copiar arquivo mantendo datas
-n	No-clobber: não sobrescrever se existir	find -exec cp -n {} /dest/ \;
-u	Copiar só se a fonte for mais nova	Sincronização incremental

ln — Hard link vs Soft link (cai na prova):

Característica	Hard Link (ln)	Soft Link (ln -s)
Aponta para	Mesmo INODE do arquivo	Caminho do arquivo original
Se original deletado	Dados ainda acessíveis	Link quebrado (dangling)
Cruzar filesystems	NÃO (mesmo filesystem)	SIM (qualquer caminho)
Para diretórios	NÃO permitido	SIM (ln -s /dir link_dir)
Identificar no ls -l	Mesmo inode (ls -li)	l no início + -> caminho

Exemplo completo: criar e verificar os dois tipos de link

```
# Hard link – mesmo inode
ln /etc/passwd /tmp/passwd_hard
ls -li /etc/passwd /tmp/passwd_hard    # MESMO número de inode nas duas linhas

# Soft link – atalho
ln -s /etc/passwd /tmp/passwd_soft
ls -la /tmp/passwd_soft
# Saída: lrwxrwxrwx ... /tmp/passwd_soft -> /etc/passwd
```

stat — informações detalhadas do arquivo:

```
stat /etc/passwd
# Saída inclui:
# File: /etc/passwd
# Size: 2847      Inode: 1234567
# Access: (0644/-rw-r--r--)  Uid: (0/root)  Gid: (0/root)
# Modify: 2025-05-01 10:00:00
```

■ Access: (0644/...) mostra a permissão NUMÉRICA — útil para diagnóstico rápido sem precisar converter rwx mentalmente.

BLOCO 8 — Compressão Standalone: gzip · bzip2 · xz · zip

Além do tar (que cria archives), o exame pode pedir para comprimir ou descomprimir arquivos individualmente sem criar um archive. Regra importante: gzip/bzip2/xz REMOVEM o original por padrão — use -k (keep) para manter.

```
# GZIP – mais rápido, compressão média
gzip arquivo.txt           # → arquivo.txt.gz (remove o original)
gzip -k arquivo.txt        # → arquivo.txt.gz (MANTÉM o original)
gunzip arquivo.txt.gz      # descomprimir
zcat arquivo.txt.gz        # ler conteúdo sem descomprimir

# BZIP2 – mais lento, compressão melhor
bzip2 -k arquivo.txt       # → arquivo.txt.bz2 (MANTÉM o original)
bunzip2 arquivo.txt.bz2    # descomprimir

# XZ – mais lento, melhor compressão
xz -k arquivo.txt          # → arquivo.txt.xz (MANTÉM o original)
unxz arquivo.txt.xz        # descomprimir

# ZIP – compatibilidade Windows
zip arquivo.zip f1 f2      # criar zip com múltiplos arquivos
zip -r pasta.zip /dir/     # zip recursivo de diretório
unzip arquivo.zip          # descomprimir
unzip -l arquivo.zip       # listar sem extrair
unzip -d /destino arquivo.zip # extrair para diretório
```

Comparativo — quando usar cada ferramenta:

Ferramenta	Extensão	Compressão	Compatibilidade
gzip	.gz	Média (rápida)	Universal Linux
bzip2	.bz2	Boa (média velocidade)	Linux/Mac
xz	.xz	Excelente (lenta)	Moderna — RHEL prefere

Ferramenta	Extensão	Compressão	Compatibilidade
zip	.zip	Média (rápida)	Windows + Linux

Regra do -k (exemplo do exame)

Tarefa: 'comprimir /root/data.txt com bzip2 mantendo o original'.

```
bzip2 -k /root/data.txt
# Gera /root/data.txt.bz2 E mantém /root/data.txt
# Sem -k: bzip2 /root/data.txt geraria o .bz2 mas REMOVERIA o data.txt
```

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
Editor disponível no exame	vim (Normal / Insert / Command)	Assumir que o nano está instalado
Substituir texto no vim	:%s/antigo/novo/g (arquivo inteiro)	:s/antigo/novo/g (só a linha atual)
Buscar sintaxe de um comando	man COMANDO + /EXAMPLES	Tentar adivinhar sem consultar
Comprimir mantendo o original	gzip -k arquivo	gzip arquivo (remove o original)
Link que sobrevive à exclusão	ln (hard link, mesmo inode)	ln -s (quebra se original for apagado)
find com muitos resultados	-exec cmd {} + (processa em lote)	-exec cmd {} \; (um por vez, mais lento)
Confirmar caminho de um binário	which dnf → caminho absoluto	Usar caminho relativo de memória
Extrair campo fixo de /etc/passwd	cut -d: -f1 (mais simples)	awk complexo para um caso simples

✓ Checklist Final

VIM

- ☐ Modos: Normal (padrão) → Insert (i, a) → Command (:). Esc volta para Normal
- ☐ :wq = salvar e sair. :q! = sair SEM salvar. ZZ = atalho para :wq
- ☐ :%s/antigo/novo/g = substituir TODAS as ocorrências no arquivo inteiro

man

- ☐ man COMANDO → /EXAMPLES → n. Funciona com semanage-port, crontab, firewall-cmd...
- ☐ man -k PALAVRA = apropos PALAVRA — buscar em todos os man pages

awk / cut

- ☐ awk -F: '{print \$1}' /etc/passwd — extrai campo 1 com delimitador ':'
- ☐ awk -F: '\$3 >= 1000 {print \$1}' /etc/passwd — filtra por condição
- ☐ cut -d: -f1 /etc/passwd — alternativa mais simples para campos fixos

Pipes / here-doc / redirecionamento

- ☐ comando | tee arquivo — saída para tela E arquivo simultaneamente
- ☐ cat > arq << 'EOF' ... EOF — criar arquivo multi-linha sem abrir editor
- ☐ 2>/dev/null — suprimir mensagens de erro (muito útil com find)

find avançado

- ☐ find /etc -name '*.conf' -o -name '*.cfg' — OR entre filtros
- ☐ find / -perm -4000 2>/dev/null — localizar binários SUID
- ☐ find /tmp -type f -empty -delete — deletar arquivos vazios diretamente
- ☐ -exec cmd {} + passa TODOS os arquivos de uma vez (mais rápido que \;)

Links, cp e stat

- ☐ ln = hard link (mesmo inode). ln -s = soft link (atalho). ls -li para ver inodes

- ☐ `cp -a` = copiar tudo: permissões + dono + links + contextos SELinux
- ☐ `stat` arquivo — ver permissão NUMÉRICA, inode, dono, datas

Compressão

- ☐ `gzip/bzip2/xz` REMOVEM o original. Use `-k` para MANTER o arquivo original
- ☐ `gunzip .gz` | `bunzip2 .bz2` | `unxz .xz` — descomprimir cada formato
- ☐ `zip -r pasta.zip /dir/` — zip recursivo para compatibilidade Windows

RHCSA EX200 — Resumo Dia 07/05

Systemd Timers · Cron · at · Chrony · Tuned · Sysctl · Journald · Processos

PROVA: 14/05

! Armadilhas do Dia

Certo	Errado
daemon-reload ANTES de enable --now	Pular daemon-reload — o systemd não enxerga os arquivos .service/.timer novos
OnBootSec=1m junto com OnUnitActiveSec=2h	Só OnUnitActiveSec sem OnBootSec — o ciclo nunca começa, timer fica inativo
OnCalendar=*-*-* 07:00:00 para horário específico	Confundir com OnCalendar=daily — daily sempre dispara à 00:00
45 0 * * * = 12:45 AM (hora 0 = meia-noite)	Usar hora 24 ou hora 12 por engano no crontab
Criar apenas /etc/cron.allow com o usuário permitido	Criar também /etc/cron.deny — desnecessário, pode causar confusão
renice -n N -p PID (muda processo já rodando)	Confundir com nice (que inicia um processo NOVO com essa niceness)
Editar /etc/sysctl.d/99-arquivo.conf + sysctl -p	Editar /proc/sys/... diretamente — funciona, mas não persiste no reboot

Tabela-Chave — Systemd Timer vs Cron vs at

Característica	Systemd Timer	Cron	at
Tipo de tarefa	Recorrente	Recorrente	Uma única vez
Job perdido (servidor desligado)	Sim (Persistent=true)	Não	Sim (roda ao ligar)
Logs	journalctl	/var/log/cron	email/syslog
Arquivo de config	/etc/systemd/system/	crontab -e	at (comando interativo)
Ativar	systemctl enable --now	automático	atd (service)
Cobrado no EX200	SIM — todos os simulados	SIM	SIM (1 simulado)

BLOCO 1 — Systemd Timers

Um Systemd Timer funciona sempre em par: um arquivo .service define O QUE executar, e um arquivo .timer define QUANDO executar. Ambos ficam em /etc/systemd/system/. A ordem de ativação é sempre: daemon-reload → enable --now. Esse tema aparece em TODOS os 6 simulados — decore o template.

Exemplo completo (S1-Q7): timer a cada 1 minuto — logar a data em um arquivo

```
# PASSO 1: criar o script executável
echo '#!/bin/bash' > /usr/local/bin/system-check.sh
echo 'date >> /var/log/system-check.log' >> /usr/local/bin/system-check.sh
chmod +x /usr/local/bin/system-check.sh

# PASSO 2: criar o .service (Type=oneshot = executa e encerra)
cat > /etc/systemd/system/system-check.service << 'EOF'
[Unit]
Description=System Check Service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/system-check.sh
EOF
```

```
# PASSO 3: criar o .timer
cat > /etc/systemd/system/system-check.timer << 'EOF'
[Unit]
Description=Run System Check every minute
[Timer]
OnCalendar=minutely
Unit=system-check.service
[Install]
WantedBy=timers.target
EOF
```

```
# PASSO 4: ativar (ORDEM OBRIGATÓRIA)
systemctl daemon-reload
systemctl enable --now system-check.timer
systemctl list-timers | grep system-check
```

■ **daemon-reload** ANTES de **enable --now**. Sem essa ordem, o **systemd** não enxerga os arquivos novos.

Exemplo completo (S3-Q5): repetição periódica com OnBootSec + OnUnitActiveSec (a cada 2h)

Para repetição periódica usa-se a combinação **OnBootSec** + **OnUnitActiveSec**. **OnBootSec=1m** define o primeiro disparo (1 min após o boot — inicia o ciclo). **OnUnitActiveSec=2h** define que, após cada execução, espera 2h para disparar de novo.

```
echo '#!/bin/bash' > /usr/local/bin/break-time.sh
echo 'echo "Break Time!" >> /tmp/break.log' >> /usr/local/bin/break-time.sh
chmod +x /usr/local/bin/break-time.sh

cat > /etc/systemd/system/break-time.service << 'EOF'
[Unit]
Description=Break Time Service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/break-time.sh
EOF

cat > /etc/systemd/system/break-time.timer << 'EOF'
[Unit]
Description=Run Break Time every 2 hours
[Timer]
OnBootSec=1m
OnUnitActiveSec=2h
Unit=break-time.service
[Install]
WantedBy=timers.target
EOF

systemctl daemon-reload
systemctl enable --now break-time.timer
```

■ **OnBootSec=1m** é obrigatório quando se usa **OnUnitActiveSec**. Sem ele, o contador de 'desde a última execução' nunca começa e o timer fica inativo.

Variações de OnCalendar já cobradas — mesmo template, muda só o horário:

Simulado	Tarefa	OnCalendar	Observação
S1-Q7	Log a cada 1 min	minutely	= *-*- * *:00
S2-Q16	Deletar temp à meia-noite	daily	+ Persistent=true
S3-Q5	Log a cada 2h	OnBootSec=1m / OnUnitActiveSec=2h	não usa OnCalendar
S4-Q5	Log às 07:00	*-*-* 07:00:00	equivale ao cron: 0 7 * * *
S5-Q16 / S6-Q19	Deletar temp às 16:15	*-*-* 16:15:00	validar com systemd-analyze calendar

■ **Persistent=true** (S2): se o servidor estava desligado no horário agendado, o job roda imediatamente quando o servidor volta. O cron simplesmente pula a execução.

Ferramenta de validação (S5-Q16):

```
systemd-analyze calendar "*-*-* 16:15:00"  
# Saída esperada:  
# Normalized form: *-*-* 16:15:00  
# Next elapse: [data e hora da próxima execução]
```

■ `systemd-analyze calendar` 'expressão' valida a sintaxe do `OnCalendar` e mostra exatamente quando o timer vai disparar — use no exame para evitar erro de digitação.

BLOCO 2 — Cron: crontab e Controle de Acesso

Cada linha do crontab segue o formato: MIN HORA DIA MES DIA_SEMANA COMANDO. O controle de acesso é feito pelos arquivos `/etc/cron.allow` e `/etc/cron.deny`.

Campo	Faixa	Exemplo
MIN	0-59	45 = minuto 45
HORA	0-23	0 = meia-noite, 7 = 07h
DIA	1-31	* = todo dia
MES	1-12	* = todo mês
DIA_SEMANA	0-7 (0=Dom)	* = qualquer

Exemplo completo (S3-Q21): cron para root às 12:45 AM

12:45 AM no formato 12h = 00:45 no formato 24h. Portanto: MIN=45, HORA=0.

```
crontab -e -u root  
# Adicionar a linha abaixo e salvar:  
45 0 * * * find /tmp -type f -empty -delete  
  
# Verificar  
crontab -l -u root
```

■ 45 0 = minuto 45, hora 0. Lembre: meia-noite é hora 0, não hora 24. 12:45 AM = 00:45 = '45 0' no crontab.

Exemplo completo (S2-Q25): permitir cron apenas para o usuário 'tom'

Se `/etc/cron.allow` existe: SOMENTE os usuários listados podem usar cron — os demais são negados automaticamente, sem precisar de `/etc/cron.deny`.

```
# Criar o arquivo com apenas tom  
echo 'tom' > /etc/cron.allow  
cat /etc/cron.allow          # tom  
  
# Testar como outro usuário (deve ser negado)  
su - sam  
crontab -l  
# Saída: You (sam) are not allowed to use this program.
```

■ A presença de `/etc/cron.allow` já é suficiente. NÃO crie `/etc/cron.deny` junto — qualquer usuário fora do `cron.allow` é negado automaticamente.

BLOCO 3 — at: Tarefa Única Agendada

O comando `at` agenda tarefas que rodam uma única vez em horário específico — diferente do `cron` (recorrente). Ideal para ações pontuais. O serviço `atd` precisa estar em execução.

Exemplo completo (S4-Q29): agendar tarefa às 23:30

```
# Garantir que o atd está rodando  
systemctl status atd  
dnf install at -y          # se não estiver instalado  
systemctl enable --now atd
```

```
# Agendar a tarefa
at 23:30
at> touch /var/tmp/late_night_check.txt
at> [Ctrl+D para salvar]
# Saída: job 1 at [data] 23:30

# Verificar a fila
atq
# Saída: 1 [data] 23:30 a root
```

■ *at* = uma vez. *cron* = recorrente. Leia o enunciado com atenção — o exame pode pedir um ou o outro dependendo da necessidade. *atrm* *N* cancela a tarefa de número *N*.

BLOCO 4 — Chrony / NTP

Chrony é a implementação de NTP usada no RHEL. O daemon *chronyd* sincroniza o relógio do sistema com servidores de tempo. *timedatectl* é a interface de linha de comando para fuso horário e NTP.

Exemplo completo (S1·Q4): timezone America/New_York + Chrony + NTP

```
# PASSO 1: definir o fuso horário
timedatectl set-timezone America/New_York
# Para listar todos: timedatectl list-timezones | grep America

# PASSO 2: instalar e habilitar o Chrony
dnf install chrony -y
systemctl enable --now chronyd

# PASSO 3: ativar sincronização NTP e verificar
timedatectl set-ntp true
timedatectl
# Deve mostrar:
# Time zone: America/New_York
# NTP synchronized: yes
```

A mesma sequência de 4 comandos se repete em todos os simulados — só muda o timezone:

Simulado	Timezone pedido
S1·Q4	America/New_York
S2·Q4	Europe/London
S4·Q6	America/Cancun
S5·Q5	America/Los_Angeles
S6·Q6	America/Chicago

■ *pool.ntp.org iburst* no */etc/chrony.conf* acelera a sincronização inicial — *iburst* envia uma rajada de 8 pacotes ao iniciar, em vez de 1.

BLOCO 5 — Tuned: Perfis de Desempenho

O *tuned* é um daemon que aplica otimizações de kernel automaticamente com base em perfis predefinidos, ajustando CPU, disco, rede e energia.

Perfil	Quando usar
throughput-performance	Servidores de dados, banco de dados — maximizar throughput
virtual-guest	VM rodando dentro de um hypervisor
powersave	Economizar energia — laptops, VMs ociosas
balanced	Desktop/laptop geral — equilíbrio entre performance e energia
accelerator-performance	GPU, HPC, latência ultra-baixa — desabilita power saving states

Exemplo completo (S1-Q17): perfil throughput-performance

```
dnf install tuned -y
systemctl enable --now tuned
tuned-adm profile throughput-performance
tuned-adm active
# Saída: Current active profile: throughput-performance
```

Exemplo — combinando dois perfis / stacking (S3-Q11, S4-Q12, S6-Q12)

O tuned suporta combinar (stackar) múltiplos perfis. O segundo perfil tem prioridade em configurações conflitantes.

```
dnf install tuned -y && systemctl enable --now tuned

# S3-Q11 — VM + economia de energia
tuned-adm profile virtual-guest powersave

# S4-Q12 — VM + latência ultra-baixa
tuned-adm profile virtual-guest accelerator-performance

tuned-adm active
# Saída: Current active profile: virtual-guest powersave
```

■ tuned-adm profile P1 P2 — espaço entre os perfis, não vírgula. O segundo perfil tem prioridade em conflitos.

BLOCO 6 — Sysctl: Parâmetros do Kernel

O sysctl permite ler e modificar parâmetros do kernel em tempo de execução. Alterações em /proc/sys/ são temporárias (perdidas no reboot); para persistência, use arquivos em /etc/sysctl.d/.

Exemplo completo (S2/S3/S4/S5/S6): habilitar packet forwarding IPv4 e IPv6, persistente

```
# PASSO 1: criar arquivo dedicado em /etc/sysctl.d/
vim /etc/sysctl.d/99-forwarding.conf
# Conteúdo do arquivo:
net.ipv4.ip_forward = 1
net.ipv6.conf.all.forwarding = 1

# PASSO 2: aplicar AGORA sem reboot
sysctl -p /etc/sysctl.d/99-forwarding.conf
# OU aplicar todos os arquivos de uma vez:
sysctl --system

# VERIFICAR
sysctl net.ipv4.ip_forward          # net.ipv4.ip_forward = 1
sysctl net.ipv6.conf.all.forwarding # net.ipv6.conf.all.forwarding = 1
```

■ O prefixo numérico (99-) define a ordem de leitura — número maior é lido por último e sobrescreve configurações anteriores. Editar /proc/sys/net/ipv4/ip_forward diretamente funciona, mas NÃO persiste após reboot.

BLOCO 7 — Journald: Logs Persistentes

O journald coleta logs de todos os serviços gerenciados pelo systemd. Por padrão (Storage=auto), os logs ficam em RAM e são perdidos no reboot. Com Storage=persistent, os logs são salvos em disco (/var/log/journal).

Exemplo completo (S1-Q21): journald persistente + filtro sshd do boot atual

```
# PASSO 1: configurar Storage=persistent
vim /etc/systemd/journald.conf
# Em [Journal], alterar/adicionar:
Storage=persistent

# PASSO 2: reiniciar e fazer flush
systemctl restart systemd-journald
journalctl --flush
ls -ld /var/log/journal
```

```
# PASSO 3: exportar logs do sshd do boot atual
journalctl -u sshd -b > /var/log/ssh_boot.log
head /var/log/ssh_boot.log
```

Filtros journalctl mais usados no exame:

Comando	Significado
journalctl -u sshd	Logs apenas do sshd
journalctl -b	Logs do boot atual
journalctl -b -1	Logs do boot anterior
journalctl --since '1 hour ago'	Logs da última hora
journalctl -u sshd -b	sshd + boot atual (combinar filtros)
journalctl --disk-usage	Espaço usado pelos logs

Exemplo completo (S5-Q25): journald persistente com limite de 100MiB

```
vim /etc/systemd/journald.conf
# Em [Journal], adicionar/descomentar:
Storage=persistent
SystemMaxUse=100M

systemctl restart systemd-journald
ls -d /var/log/journal
journalctl --disk-usage
```

■ SystemMaxUse define o espaço máximo total dos logs em disco. Ao atingir o limite, logs antigos são deletados automaticamente (rotação).

BLOCO 8 — Processos: nice, renice, kill, signals

nice e renice controlam a prioridade de CPU de processos. A niceness vai de -20 (maior prioridade) a +19 (menor prioridade); o padrão é 0. Apenas root pode usar valores negativos.

Sinal	Número	Ação
SIGTERM	15	Encerramento gracioso — processo pode limpar recursos
SIGKILL	9	Encerramento forçado — não pode ser ignorado
SIGINT	2	Interrupção (equivalente ao Ctrl+C)

Exemplo completo (S1-Q22): sleep 1000 em background, niceness=5, SIGTERM

```
# PASSO 1: iniciar em background
sleep 1000 &
# Shell imprime ex: [1] 12345 (número do job e PID)

# PASSO 2: encontrar o PID (se não anotou)
pgrep -a sleep
# OU: jobs -l

# PASSO 3: mudar niceness para 5
renice -n 5 -p PID
ps -o pid,ni,comm -p PID      # verificar NI=5

# PASSO 4: enviar SIGTERM e confirmar
kill PID
jobs                          # deve mostrar: [1]+ Terminated
```

■ renice MUDA a niceness de um processo já em execução. nice iniciaria um NOVO processo com essa niceness — não confundir os dois.

Variações já cobradas — mesmo padrão, muda o valor de niceness:

Simulado	Processo	Niceness	Sinal
S1-Q22	sleep 1000	5	SIGTERM
S2-Q30	sleep 5000	15	SIGTERM
S3-Q25	sleep 1000	10	SIGTERM

Exemplo completo (S4-Q23): sleep 10000 em background, SIGINT (sem renice)

```
sleep 10000 &
pgrep -a sleep          # localizar o PID

kill -2 PID
# OU equivalente:
kill -SIGINT PID

jobs                    # Terminated
```

■ nice INICIA um processo com uma niceness. renice MUDA a niceness de um processo já rodando. Confundir os dois é erro clássico no exame.

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
Ativar timer novo	daemon-reload + enable --now	Só enable --now (arquivo não é lido)
Repetição periódica	OnBootSec=1m + OnUnitActiveSec=2h	Só OnUnitActiveSec (nunca dispara)
Horário fixo no timer	OnCalendar=*-*-* 07:00:00	OnCalendar=daily (sempre 00:00)
Cron às 12:45 AM	45 0 * * *	45 24 * * * (hora 24 não existe)
Restringir cron a 1 usuário	Só /etc/cron.allow com o nome	Criar cron.allow E cron.deny juntos
Mudar prioridade de processo ativo	renice -n N -p PID	nice -n N (inicia processo novo)
Persistência de sysctl	/etc/sysctl.d/99-arquivo.conf + sysctl -p	Editar /proc/sys/ direto (não persiste)
Job perdido com servidor desligado	Persistent=true no .timer	Cron — simplesmente pula a execução

✓ Checklist Final

Systemd Timers

- ☐ Par .service (Type=oneshot + ExecStart) e .timer (OnCalendar + Unit=)
- ☐ daemon-reload ANTES de enable --now — ordem obrigatória
- ☐ systemctl list-timers para ver a próxima execução
- ☐ OnCalendar: minutely, daily, *-*-* 07:00:00, *-*-* 16:15:00
- ☐ OnBootSec=1m + OnUnitActiveSec=2h para repetição a cada 2h
- ☐ Persistent=true para rodar job perdido ao religar o servidor

Cron e at

- ☐ crontab formato: MIN HORA DIA MES DIASEM COMANDO
- ☐ 45 0 * * * = 12:45 AM. 0 7 * * * = 07:00
- ☐ /etc/cron.allow: só os listados podem usar cron
- ☐ at 23:30 → prompt at> → Ctrl+D para salvar. atq para verificar

Chrony / Tuned

- ☐ timedatectl set-timezone + dnf install chrony + enable --now chronyd
- ☐ timedatectl set-ntp true + verificar NTP synchronized: yes
- ☐ tuned-adm profile NOME. Combinar: tuned-adm profile P1 P2

Sysctl / Journald

- ☐ /etc/sysctl.d/99-forwarding.conf + sysctl -p arquivo
- ☐ journald: Storage=persistent + SystemMaxUse=100M em journald.conf
- ☐ journalctl -u sshd -b > /var/log/ssh_boot.log

Processos

- ☐ nice -n N cmd & (iniciar). renice -n N -p PID (modificar)
- ☐ kill = SIGTERM(15). kill -9 = SIGKILL. kill -2 = SIGINT

RHCSA EX200 — Resumo Dia 06/05

Scripts Shell · grep · sed · find · Variáveis de Ambiente

PROVA: 14/05

! Armadilhas do Dia

Certo	Errado
chmod +x sempre após criar o script	Esquecer chmod +x — script correto, mas eliminatório porque não roda
#!/bin/bash na linha 1, sem espaço antes do #	Shebang ausente ou mal posicionado — sistema tenta executar como binário
\$(id -u \$USER) — retorna só o número do UID	\$(id \$USER) — retorna a linha inteira (uid=0(root) gid=...)
for NUM in "\$@" (com aspas)	for NUM in \$@ (sem aspas) — quebra argumentos com espaço
while read VAR < arquivo (redirecionamento)	cat arquivo while read VAR — pipe cria subshell, variáveis somem
comando > arquivo 2>&1 (ordem correta)	comando 2>&1 > arquivo — stderr ainda vai para o terminal
find ... -perm -4000 (pelo menos este bit)	find ... -perm 4000 (exige permissão EXATAMENTE 4000)

Tabela-Chave — Variáveis Especiais do Shell

Sintaxe	Significado
\$1 \$2 \$3	argumentos posicionais (1º, 2º, 3º)
\$@	todos os argumentos como lista
\$#	número de argumentos
\$()	substituição de comando
\$(())	aritmética inteira
-gt -lt -eq	comparadores numéricos: maior, menor, igual

BLOCO 1 — Estrutura Base de Todo Script

Todo script no EX200 segue exatamente a mesma estrutura. Internalize como um ritual: shebang → lógica → chmod +x → testar.

```
#!/bin/bash
# Linha 1: SEMPRE shebang #!/bin/bash

# Variáveis
NOME="valor"
RESULTADO=$(comando)          # capturar saída de comando
SOMA=$((NUM1 + NUM2))         # aritmética inteira

# Argumentos posicionais
$1 $2 $3                      # primeiro, segundo, terceiro argumento
"$@"                          # TODOS os argumentos como lista
$#                             # número de argumentos

# Criar, tornar executável e testar
vim /usr/local/bin/meu_script.sh
chmod +x /usr/local/bin/meu_script.sh
/usr/local/bin/meu_script.sh arg1 arg2
```

- `chmod +x` é eliminatório. Sem ele o avaliador não consegue executar o script para corrigir — perde o ponto mesmo com a lógica certa.
- Sem o shebang `#!/bin/bash` o sistema tenta executar como binário e falha. Sempre na linha 1, sem espaço antes do #.

BLOCO 2 — Argumentos: \$1, \$2, @\$

Exemplo completo (S1-Q11): flipargs.sh — inverter dois argumentos

```
#!/bin/bash
echo "$2 $1"

chmod +x /usr/local/bin/flipargs.sh

# Testar
/usr/local/bin/flipargs.sh hello world
# Output: world hello
```

- \$1 = primeiro argumento, \$2 = segundo. Inverter a ordem no echo é tudo que precisamos.

Exemplo completo (S1-Q30): checkfile.sh — verificar se arquivo existe

```
#!/bin/bash
if [ -f "$1" ]; then
    echo "File exists."
else
    echo "File missing."
fi

chmod +x /usr/local/bin/checkfile.sh

# Testar
/usr/local/bin/checkfile.sh /etc/passwd      # Output: File exists.
/usr/local/bin/checkfile.sh /etc/naoexiste   # Output: File missing.
```

Flags de teste — Referência:

Flag	Significado	Exemplo
-f	É arquivo regular	[-f "/etc/passwd"]
-d	É diretório	[-d "/tmp"]
-e	Existe (arquivo ou dir)	[-e "/caminho"]
-z	String vazia	[-z "\$VAR"]
-n	String não vazia	[-n "\$VAR"]

- Aspas em "\$1" evitam erros com nomes de arquivo com espaços. Sempre cite variáveis dentro de [].

BLOCO 3 — Loop for com Lista Fixa

Exemplo completo (S1-Q24): user_audit.sh — imprimir nome e UID de cada usuário

```
#!/bin/bash
for USER in root adm ftp
do
    USERID=$(id -u $USER)
    echo "User $USER has ID $USERID"
done

chmod +x /usr/local/bin/user_audit.sh

# Output:
# User root has ID 0
# User adm has ID 3
# User ftp has ID 14
```

- `$(id -u $USER)` retorna só o número do UID. `$(id $USER)` retorna `uid=0(root) gid=...` — texto completo que não é o pedido.
- `$()` é substituição de comando: executa o comando em subshell e captura a saída. Forma moderna de backticks — mais legível.

Variação (S3-Q26): user-list.sh — imprimir mensagem para cada usuário

```
#!/bin/bash
for USER in root bin daemon
do
    echo "Processing user: $USER"
done

chmod +x /usr/local/bin/user-list.sh
```

Template mental do for com lista:

```
for VARIÁVEL in item1 item2 item3
do
    # corpo do loop — $VARIÁVEL tem o item atual
done
```

BLOCO 4 — Loop for + \$@ + if — Somar Positivos

Destaque: este é o script mais complexo do exame — já caiu em dois simulados (S5-Q13 e S6-Q16). Combina: `$@` para número ilimitado de argumentos + `for` para iterar + `if` para filtrar + `$( )` para somar. Pratique até memorizar.

Exemplo completo (S5-Q13 / S6-Q16): sum.sh — somar apenas argumentos maiores que 0

```
#!/bin/bash
TOTAL=0
for NUM in "$@"
do
    if [ "$NUM" -gt 0 ]; then
        TOTAL=$((TOTAL + NUM))
    fi
done
echo "The sum is $TOTAL"

chmod +x /sum.sh

# Testar
/sum.sh 10 20 -5 30      # Output: The sum is 60 (ignora -5)
/sum.sh 10 20 -5         # Output: The sum is 30
```

Cada elemento deste script, explicado:

- `TOTAL=0` — inicializar o acumulador ANTES do loop; sem isso a aritmética pode falhar.
- `"$@"` — expande para todos os argumentos como itens separados; com aspas preserva espaços.
- `-gt 0` — greater than 0 — filtra negativos e zero.
- `=$((TOTAL + NUM))` — aritmética inteira; resultado atribuído à mesma variável.

Tabela de comparadores numéricos:

Operador	Significado	Exemplo
-gt	maior que	["\$N" -gt 0]
-lt	menor que	["\$N" -lt 10]
-eq	igual	["\$A" -eq "\$B"]
-ne	diferente	["\$A" -ne 0]
-ge	maior ou igual	["\$N" -ge 1]
-le	menor ou igual	["\$N" -le 100]

BLOCO 5 — Loop while — Ler Arquivo Linha por Linha

Exemplo completo (S4-Q22): process_lines.sh — processar arquivo com DATA NOME

```
# Primeiro criar o arquivo de teste
cat > /root/dates.txt << EOF
2000-05-01 Sam
2002-06-15 Sue
2004-12-29 Sarah
2005-05-10 Mike
EOF

#!/bin/bash
while read DATE NAME
do
    echo "Name: $NAME was born on $DATE"
done < /root/dates.txt

chmod +x /usr/local/bin/process_lines.sh

# Output:
# Name: Sam was born on 2000-05-01
# Name: Sue was born on 2002-06-15
# Name: Sarah was born on 2004-12-29
# Name: Mike was born on 2005-05-10
```

- done < /root/dates.txt redireciona o arquivo como entrada do loop. while read VAR1 VAR2 divide cada linha pelos espaços.
- NÃO use cat arquivo | while read (o pipe cria subshell — variáveis modificadas dentro não ficam visíveis fora). Use redirecionamento com < .

BLOCO 6 — read Interativo + Aritmética \$(())

Exemplo completo (S3-Q12): sum.sh interativo — pedir dois números ao usuário

```
#!/bin/bash
echo "Enter the first number"
read num1
echo "Enter the second number"
read num2
sum=$((num1 + num2))
echo "The result of addition=$sum"

chmod +x /usr/local/bin/sum.sh

# Execução interativa:
# /usr/local/bin/sum.sh
# Enter the first number
# 5
# Enter the second number
# 3
# The result of addition=8
```

Operações com \$(()):

Expressão	Operação
\$((A + B))	Soma
\$((A - B))	Subtração
\$((A * B))	Multiplicação
\$((A / B))	Divisão inteira
\$((A % B))	Módulo (resto)

- Leia o enunciado: 'pede ao usuário' → read interativo. 'aceita argumentos' → \$1 \$2. Confundir os dois = script errado.

BLOCO 7 — case + tr — Case-Insensitive

Exemplo completo (S2-Q9): yes-no.sh — resposta diferente para yes/no/outros

```
#!/bin/bash
# Converter para minúsculo para comparação case-insensitive
INPUT=$(echo "$1" | tr '[:upper:]' '[:lower:]')

case "$INPUT" in
    yes)
        echo "That is nice"
        ;;
    no)
        echo "I am sorry"
        ;;
    *)
        echo "Unknown argument"
        ;;
esac

chmod +x /usr/local/bin/yes-no.sh

# Testar
/usr/local/bin/yes-no.sh yes      # That is nice
/usr/local/bin/yes-no.sh YES      # That is nice (case-insensitive!)
/usr/local/bin/yes-no.sh no       # I am sorry
/usr/local/bin/yes-no.sh maybe    # Unknown argument
```

Anatomia do case:

- `tr '[:upper:]' '[:lower:]'` — converte tudo para minúsculas antes de comparar.
- `case "$INPUT" in` — abre o case; variável sempre entre aspas.
- `;;` — termina cada bloco — obrigatório, como um `break`. `*)` é o default, equivalente ao `else`. `esac` fecha o case ('case' ao contrário).
- case é mais limpo que `if/elif/else` para múltiplas opções do mesmo valor. Esquecer `;;` ou `esac` causa erro de sintaxe.

BLOCO 8 — Scripts com find — SUID e Auditoria

Exemplo completo (S3-Q28): find_suid.sh — listar arquivos SUID em /usr/bin

```
#!/bin/bash
find /usr/bin -user root -perm -4000

chmod +x /root/find_suid.sh
# Output: /usr/bin/sudo, /usr/bin/passwd, etc.
```

- `-perm -4000`: o hífen significa 'pelo menos este bit ativo'. Sem hífen (`-perm 4000`) exige permissão EXATAMENTE 4000.

Exemplo completo (S4-Q27): audit_suid.sh — copiar SUID < 10MiB para /root/audit_results/

```
#!/bin/bash
mkdir -p /root/audit_results
find /usr -type f -perm /u=s -size -10M -exec cp -a {} /root/audit_results/ \;

chmod +x /usr/local/bin/audit_suid.sh
ls /root/audit_results/
```

- `cp -a` preserva TODOS os atributos, inclusive o bit SUID. `-size -10M`: hífen = menor que; sem sinal = exato; + = maior que.

Variação (S5-Q29): audit_suid.sh — salvar lista de SUID < 5MiB em arquivo

```
#!/bin/bash
find /usr -type f -perm /4000 -size -5M > /root/suid_files.txt

chmod +x /usr/local/bin/audit_suid.sh
cat /root/suid_files.txt
```

Flags do find — Referência:

Flag	Significado	Exemplo
-perm -4000	SUID ativo (pelo menos)	find /usr/bin -perm -4000
-perm /u=s	SUID ativo (forma simbólica)	find /usr -perm /u=s
-perm /4000	SUID ativo (alternativa)	find /usr -perm /4000
-size -10M	Menor que 10MB	-size -5M (menor que 5MB)
-type f	Apenas arquivos regulares	-type d (diretórios)
-user root	Dono é root	-user alice
-exec CMD {} \;	Executar CMD em cada resultado	-exec cp -a {} /dest/ \;

BLOCO 9 — grep · sed · Redirecionamento

Referência rápida:

```
# GREP — busca
grep "texto" arquivo          # busca simples
grep -i "texto" arquivo       # case-insensitive
grep -r "texto" /diretorio    # recursivo
grep "^Host" arquivo          # linhas que COMEÇAM com "Host"
grep -v "texto" arquivo       # linhas que NÃO contêm
grep '-0[56]-' arquivo        # character class [56] = 5 ou 6
grep -E '-(05|06)-' arquivo   # regex estendida (OR)

# SED — substituição
sed 's/antigo/novo/' arquivo  # 1ª ocorrência por linha
sed 's/antigo/novo/g' arquivo # TODAS (g = global)
sed 's/sam/Sam/g' letter > newletter # resultado em novo arquivo

# REDIRECIONAMENTO
comando > arquivo            # SOBRESCREVE
comando >> arquivo           # ACRESCENTA
comando 2> arquivo           # só stderr
comando > arq 2>&1           # stdout + stderr
comando &> arquivo           # stdout + stderr (atalho Bash)
```

Exemplo (S1-Q25): grep — linhas começando com 'Host' em sshd_config

```
# ^Host ancora no início da linha — exclui comentários automaticamente
grep -i '^Host' /etc/ssh/sshd_config > /root/ssh_hosts.txt
cat /root/ssh_hosts.txt
```

■ ^Host com ^ casa apenas linhas que COMEÇAM com Host. Comentários começam com # — não são incluídos.

Exemplo (S3-Q19): sed — substituir 'sam' por 'Sam' em todas as ocorrências

```
echo 'sam is here. hello sam. sam again.' > letter
sed 's/sam/Sam/g' letter > newletter
cat newletter    # Sam is here. hello Sam. Sam again.
cat letter       # sam is here. hello sam. sam again. (original intacto!)
```

■ Sem 'g': só a 1ª ocorrência por linha. sed não modifica o arquivo original — o > captura a saída em um novo arquivo.

Exemplo (S4-Q19): redirecionar stdout E stderr para o mesmo arquivo

```
ls /root /fake_directory > /var/tmp/ls_output.txt 2>&1
# OU forma curta Bash:
ls /root /fake_directory &> /var/tmp/ls_output.txt
cat /var/tmp/ls_output.txt # tem a listagem E o erro
```

■ Ordem importa: '> arquivo 2>&1' funciona. '2>&1 > arquivo' NÃO — o stderr vai para o terminal.

Exemplo (S4-Q21): grep com character class [56] — filtrar meses 05 e 06

```
cat > /root/dates.txt << EOF
2000-05-01 Sam
2002-06-15 Sue
2004-12-29 Sarah
2005-05-10 Mike
EOF

grep '-0[56]-' /root/dates.txt > /root/summer_birthdays.txt
# OU com regex estendida:
grep -E '-(05|06)-' /root/dates.txt > /root/summer_birthdays.txt
cat /root/summer_birthdays.txt # Sam (05), Sue (06), Mike (05)
```

■ [56] casa '5' ou '6'. Os hífens ao redor (-0[56]-) garantem que está no campo do mês — não confunde com o ano (2005).

BLOCO 10 — Variáveis de Ambiente

Exemplo completo (S3-Q24): export — variável na sessão atual e processos filhos

```
export EXAM_TYPE="RHCSA"

# Capturar saída de comando em variável
TODAY=$(date +%F) # ex: 2025-05-06

# Verificar em subshell (processo filho)
bash -c 'echo $EXAM_TYPE' # deve mostrar: RHCSA
echo $TODAY # data atual
```

■ export torna a variável disponível para processos filhos. Sem export, scripts rodados a partir desta sessão não a verão.

Exemplo completo (S4-Q20): /etc/profile.d/ — variável global para todos os usuários

```
vim /etc/profile.d/class_var.sh
# Conteúdo:
export CLASS="RHCSA"

# Verificar (simular login de outro usuário)
su - sarah
echo $CLASS # deve mostrar: RHCSA
```

■ Qualquer arquivo .sh em /etc/profile.d/ é executado no login de todos os usuários. Mais limpo e seguro que editar /etc/profile diretamente.

Exemplo completo (S5-Q24): /etc/environment — variável global inclusive via SSH

```
vim /etc/environment
# Conteúdo (sem 'export', sem lógica shell):
VAR="RHCSA Prac Five"

# Verificar (logout e login, ou:)
su -
echo $VAR # RHCSA Prac Five
```

■ Não é script shell — não usa export, não suporta lógica. Lido pelo PAM para todos os tipos de login, inclusive SSH.

Comparativo — quando usar cada abordagem:

Abordagem	Usa export?	Suporta lógica?	Funciona SSH?	Quando usar
export VAR=...	Sim	Sim	Não persiste	Sessão atual e filhos
/etc/profile.d/	Sim	Sim	Shell interativo	Variáveis/scripts globais com lógica
/etc/environment	Não	Não	Sim (PAM)	Variáveis estáticas para todos

Tabela de Referência Rápida — Comparadores Bash

Comparador	Significado	Exemplo
-eq	igual (números)	["\$A" -eq "\$B"]
-ne	diferente (números)	["\$A" -ne 0]
-gt	maior que	["\$NUM" -gt 0]
-lt	menor que	["\$NUM" -lt 10]
-ge	maior ou igual	["\$NUM" -ge 1]
-le	menor ou igual	["\$NUM" -le 100]
=	igual (strings)	["\$STR" = "sim"]
!=	diferente (strings)	["\$STR" != ""]
-f	é arquivo regular	[-f "/etc/passwd"]
-d	é diretório	[-d "/tmp"]
-z	string vazia	[-z "\$VAR"]
##	número de argumentos	["\$#" -ne 2]

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
Executar script	chmod +x arquivo	Esquecer chmod +x (não roda)
Argumentos com espaço	for NUM in "\$@" (com aspas)	for NUM in \$@ (sem aspas, quebra)
UID de usuário	\$(id -u \$USER)	\$(id \$USER) — retorna texto completo
Ler arquivo linha a linha	while read VAR < arquivo	cat arquivo while read VAR (subshell)
Redirecionar stdout+stderr	comando > arquivo 2>&1	comando 2>&1 > arquivo (ordem errada)
SUID exato vs pelo menos	-perm -4000	-perm 4000 (exige valor exato)
sed substituir todas ocorrências	sed 's/x/y/g'	sed 's/x/y/' (só a 1ª ocorrência)
Variável visível em subshell	export VAR=valor	VAR=valor (sem export, filho não vê)

✓ Checklist Final

Estrutura Base

- ☐ Todo script começa com #!/bin/bash e termina com chmod +x
- ☐ \$() para capturar saída de comando,=\$(()) para aritmética

Argumentos

- ☐ flipargs.sh: echo "\$2 \$1" (inverter argumentos)
- ☐ checkfile.sh: if [-f "\$1"] then/else — aspas em \$1 obrigatórias

Loop for lista

- ☐ user_audit.sh: for USER in lista + USERID=\$(id -u \$USER)
- ☐ \$(id -u \$USER) retorna número — \$(id \$USER) retorna texto completo

for + \$@ + if

- ☐ sum.sh: TOTAL=0 antes do loop + for NUM in "\$@" + if ["\$NUM" -gt 0]
- ☐ "\$@" com aspas preserva argumentos — -gt = greater than

while + arquivo

- ☐ process_lines.sh: while read VAR1 VAR2 + done < arquivo

- ☐ NÃO usar pipe (cat | while) — subshell perde variáveis

read + aritmética

- ☐ sum.sh interativo: echo + read + \$((num1 + num2))
- ☐ Leia o enunciado: 'pede ao usuário' = read vs 'aceita argumentos' = \$1 \$2

case + tr

- ☐ yes-no.sh: tr '[:upper:]' '[:lower:]' antes do case
- ☐ Anatomia: case/in/opção/;;/*)/esac — esquecer ;; causa erro

find SUID

- ☐ find_suid.sh: find /usr/bin -user root -perm -4000
- ☐ -perm -4000 (hífen = pelo menos). -size -10M (hífen = menor que)
- ☐ -exec cp -a {} /dest/ \; preserva atributos incluindo SUID

grep • sed • redir

- ☐ grep -i '^Host' arquivo > saida.txt (^ ancora no início)
- ☐ sed 's/sam/Sam/g' arquivo > novo (g=todas, original intacto)
- ☐ > arquivo 2>&1 ou &> arquivo (ordem importa no 2>&1!)
- ☐ grep '[56]' para character class — grep -E '(05|06)' para OR

Variáveis Ambiente

- ☐ export VAR=valor para sessão atual e processos filhos
- ☐ /etc/profile.d/*.sh para variável global com suporte a lógica
- ☐ /etc/environment para variável global sem export (funciona SSH/PAM)

RHCSA EX200 — Resumo Dia 05/05

SELinux Completo · Firewall · Apache Porta Não-padrão

PROVA: 14/05

4 Pilares do SELinux no EX200

Pilar	Comando-chave
Modo global	<code>setenforce + /etc/selinux/config</code>
Contexto de arquivo	<code>semanage fcontext + restorecon</code>
Boolean	<code>setsebool -P BOOL 1</code>
Porta não-padrão	<code>semanage port -a -t TIPO -p tcp PORTA</code>

Regra dos Dois Passos — vale para SELinux E Firewall

Camada	Ativo agora	Persistente (reboot)
SELinux	<code>setenforce 1</code>	<code>SELINUX=enforcing</code> no config
Firewall	<code>firewall-cmd (sem --permanent)</code>	<code>--permanent + --reload</code>

BLOCO 1 — SELinux: Modo Enforcing / Permissive

A Regra dos Dois Passos: SELinux tem duas camadas independentes que precisam ser configuradas juntas. `setenforce` muda o modo **agora** (RAM — não sobrevive ao reboot). `/etc/selinux/config` define o modo que o kernel carrega no **próximo boot**. Configurar só um dos dois perde pontos no exame.

```
# Verificar modo atual
getenforce      # Enforcing / Permissive / Disabled
sestatus        # informações detalhadas

# PASSO 1: mudar AGORA (runtime)
setenforce 1    # → Enforcing
setenforce 0    # → Permissive

# PASSO 2: tornar PERSISTENTE
vim /etc/selinux/config
# SELINUX=enforcing (ou: permissive)

# Confirmar
getenforce
```

Exemplo completo (S2-Q26 / S3-Q18): modo `enforcing`

```
setenforce 1
vim /etc/selinux/config
# SELINUX=enforcing
getenforce  # Enforcing
```

Se o sistema estava em `disabled`, `setenforce 1` **não funciona** — é necessário reboot para o kernel carregar os módulos do SELinux.

Simulado	Modo pedido	Runtime	Config file
S2-Q26	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>
S3-Q18	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>
S4-Q16	permissive	<code>setenforce 0</code>	<code>SELINUX=permissive</code>
S5-Q21	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>
S6-Q25	enforcing	<code>setenforce 1</code>	<code>SELINUX=enforcing</code>

BLOCO 2 — SELinux: Contexto de Arquivos

Por que importa: o SELinux atribui uma etiqueta (contexto) a cada arquivo. Ao criar um diretório fora dos caminhos padrão, o contexto fica errado e serviços como o Apache não conseguem acessá-lo mesmo com `chmod` correto. Dois comandos são **sempre** necessários em sequência: `semanage fcontext` escreve a regra no banco de políticas; `restorecon` aplica a regra do banco ao disco.

```
# Ver contexto atual
ls -Z /arquivo
ls -Zd /diretorio

# PASSO 1: regra no banco de políticas
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)"
# -a = add | -t = tipo | regex "/web(/.*)" = /web E tudo dentro

# PASSO 2: aplicar a regra ao disco
restorecon -Rv /web
# -R = recursivo | -v = verbose

# Verificar
ls -Z /web/content/html/index.html # deve conter: httpd_sys_content_t
```

Exemplo completo (S2-Q23): criar `/web/content/html/index.html` , contexto `httpd_sys_content_t` , persistente

```
mkdir -p /web/content/html
touch /web/content/html/index.html
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
restorecon -Rv /web
ls -Z /web/content/html/index.html
```

O regex `"/web(/.*)?"` cobre `/web` sozinho e tudo dentro. O `?` torna o grupo opcional.

Variação (S5-Q27): `/backup` com o mesmo contexto de `/home` (`home_root_t`)

```
mkdir /backup
ls -Zd /home # confirmar contexto de referência: home_root_t
semanage fcontext -a -t home_root_t "/backup(/.*)?"
restorecon -Rv /backup
ls -Zd /backup # deve mostrar home_root_t
```

`semanage fcontext sem restorecon` = regra no banco, arquivo no disco **errado**. `restorecon sem semanage` = muda agora, mas some no próximo relabel.

Contexto	Uso
<code>httpd_sys_content_t</code>	Conteúdo servido pelo Apache (equivalente a <code>/var/www/html</code>)
<code>home_root_t</code>	Diretórios home (<code>/home</code>)
<code>ssh_home_t</code>	Arquivos SSH (<code>~/.ssh</code>)
<code>tmp_t</code>	Diretório <code>/tmp</code>

BLOCO 3 — SELinux: Booleans

O que são: interruptores on/off na política SELinux — ativam comportamentos específicos sem alterar a política inteira. Ex.: mesmo com `chmod 755` correto, o SELinux bloqueia o Apache de ler arquivos NFS por padrão; o boolean `httpd_use_nfs` é a chave.

```
# Encontrar o boolean correto
semanage boolean -l | grep nfs
semanage boolean -l | grep ftp
getsebool -a | grep http

# Ativar — SEMPRE com -P no exame
setsebool -P httpd_use_nfs 1 # Apache lê arquivos NFS
setsebool -P ftpd_full_access 1 # FTP acesso total
setsebool -P httpd_can_network_connect 1 # Apache conecta à rede

# Desativar
setsebool -P httpd_use_nfs 0

# Verificar
getsebool httpd_use_nfs
# Output: httpd_use_nfs --> on
```

Exemplo completo (S5-Q26): Apache + boolean `httpd_use_nfs=on` persistente + firewall http

```
dnf install httpd -y
systemctl enable --now httpd
firewall-cmd --permanent --add-service=http && firewall-cmd --reload
setsebool -P httpd_use_nfs 1
getsebool httpd_use_nfs # httpd_use_nfs --> on
```

Variação (S6-Q29): vsftpd + boolean `ftpd_full_access=on` persistente + firewall ftp

```
dnf install vsftpd -y
systemctl enable --now vsftpd
firewall-cmd --permanent --add-service=ftp && firewall-cmd --reload
setsebool -P ftpd_full_access 1
getsebool ftpd_full_access # ftpd_full_access --> on
```

`-P` é **obrigatório** para persistência. Sem `-P` , o boolean volta ao padrão no próximo reboot — tarefa falha. Para achar o boolean certo: `semanage boolean -l | grep TERMO` — a descrição explica o que cada um faz.

BLOCO 4 — SELinux: Porta Não-Padrão (Apache porta 82)

Por que três passos: para usar porta não-padrão, três bloqueios independentes precisam ser resolvidos: 1) o serviço precisa escutar na nova porta; 2) o SELinux precisa liberar aquela porta para o tipo do serviço (`semanage port`); 3) o firewall precisa liberar a porta (`firewall-cmd`). Resolver só um ou dois não basta — o serviço fica inacessível.

Exemplo completo (S1-Q27): Apache na porta 82

```
# 1. Configuração do Apache
vim /etc/httpd/conf/httpd.conf
# Listen 80 -> Listen 82

# 2. SELinux — adicionar porta 82 ao tipo http_port_t
semanage port -a -t http_port_t -p tcp 82

# 3. Firewall — liberar porta 82
firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --reload

# 4. Reiniciar Apache
```

```
systemctl restart httpd

# Verificar
semanage port -l | grep http_port_t      # deve listar porta 82
curl http://localhost:82                  # deve responder
```

Dois bloqueios independentes: SELinux (`semanage port`) e Firewall (`firewall-cmd`). Resolver só um não basta.

Referência — `semanage port` :

```
semanage port -l | grep http_port_t      # portas do Apache
semanage port -l | grep ssh_port_t       # portas do SSH
semanage port -l | grep ftp_port_t       # portas do FTP

semanage port -a -t http_port_t -p tcp 8080 # adicionar porta nova
semanage port -a -t ssh_port_t -p tcp 2222

semanage port -m -t http_port_t -p tcp 82  # modificar porta já existente com outro tipo

# forma longa (útil para copiar do man no exame)
semanage port --add --type http_port_t --proto tcp 82
```

`man semanage-port` → seção EXAMPLES tem o comando exato — use no exame para não errar a sintaxe.

BLOCO 5 — SELinux: Domínio Permissivo

Domínio permissivo vs modo global: `setenforce 0` coloca o **sistema inteiro** em permissive — todos os processos ignoram o SELinux. `semanage permissive -a httpd_t` coloca **apenas** o processo httpd em permissive — o resto do sistema continua protegido em Enforcing. Muito mais seguro para depurar um serviço específico.

Exemplo completo (S4-Q26): `httpd_t` em modo permissivo

```
semanage permissive -a httpd_t
semanage permissive -l      # deve listar httpd_t
getenforce                  # Enforcing (para todo o resto)

# Para remover:
semanage permissive -d httpd_t
```

`semanage permissive -a DOMINIO` é cirúrgico. Apenas aquele processo fica permissivo — a segurança geral do sistema é mantida.

BLOCO 6 — Firewall (firewall-cmd padrão)

A Regra dos Dois Passos do Firewall: `--permanent` escreve a regra em disco — não afeta o que está ativo agora. `--reload` carrega o arquivo com as regras permanentes para o kernel — agora está ativo. `--permanent` sem `--reload` : salvo mas não ativo. `--reload` sem `--permanent` : ativo agora, mas perde no reboot.

```
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service={http,https}  # múltiplos de uma vez
firewall-cmd --permanent --add-service=ftp
firewall-cmd --permanent --add-service=ssh

firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --permanent --add-port=2222/tcp

firewall-cmd --reload      # SEMPRE após --permanent

# Verificar
firewall-cmd --list-all
firewall-cmd --list-services
firewall-cmd --list-ports
```

Exemplo completo (S1-Q8): Apache padrão + firewall http

```
dnf install httpd -y
echo "Welcome to RHEL 10" > /var/www/html/index.html
systemctl enable --now httpd
firewall-cmd --permanent --add-service=http
firewall-cmd --reload
curl http://localhost      # verificar conteúdo
```

Simulado	Conteúdo do index.html	Serviços liberados
S1-Q8	"Welcome to RHEL 10"	http
S2-Q8	"Hello World!"	{http,https}
S3-Q13	"Welcome to the RHCSA Practice Exam!"	{http,https}
S4-Q13	"Welcome to the RHCSA Exam"	{http,https}
S5-Q11	"Hello Guys!"	{http,https}
S6-Q13	"Hello World!"	{http,https}

A partir do S2, o padrão do exame é liberar `{http,https}` de uma vez só: `firewall-cmd --permanent --add-service={http,https}` .

BLOCO 7 — Firewall: Rich Rules

Quando usar: Rich Rules permitem lógica condicional que `--add-service` não consegue — liberar um serviço apenas para uma origem específica e rejeitar para todos os outros. Útil para restringir SSH ou HTTP a uma rede corporativa.

```
# Liberar serviço apenas para rede específica
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    source address="192.168.1.0/24" service name="http" accept'

# Rejeitar para todos os outros
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    service name="http" reject'

firewall-cmd --reload
firewall-cmd --list-rich-rules
```

Exemplo completo (S5-Q28): HTTP permitido só na rede 192.168.1.0/24 , rejeitado nas demais

```
firewall-cmd --permanent --remove-service=http 2>/dev/null    # remover regra genérica, se existir

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    source address="192.168.1.0/24" service name="http" accept'

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    service name="http" reject'

firewall-cmd --reload
firewall-cmd --list-rich-rules
```

Variação (S6-Q27): mesmo padrão aplicado ao SSH

```
firewall-cmd --permanent --remove-service=ssh 2>/dev/null

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    source address="192.168.1.0/24" service name="ssh" accept'

firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
    service name="ssh" reject'

firewall-cmd --reload
firewall-cmd --list-rich-rules
```

Se `--add-service=http` (ou `ssh`) ainda estiver ativo, ele permite acesso de **todos**, ignorando a rich rule de `reject` — remova o genérico primeiro. Ordem importa: `accept` para rede específica deve estar antes do `reject` geral — o firewalld processa de cima para baixo.

🏠 Tabela Comparativa — 4 Pilares do SELinux

Situação	Ferramenta	Comando-chave	Persistente?
Mudar modo global	<code>setenforce</code> + <code>config</code>	<code>setenforce 1</code> + <code>SELINUX=enforcing</code>	Só com <code>config file</code>
Contexto de arquivo	<code>semanage fcontext</code> + <code>restorecon</code>	<code>semanage fcontext -a -t TIPO "REGEX" + <code>restorecon -Rv /dir</code></code>	Sim (banco de política)
Ligar/desligar feature	<code>setsebool</code>	<code>setsebool -P BOOL 1</code>	Só com <code>-P</code>
Porta não-padrão	<code>semanage port</code>	<code>semanage port -a -t TIPO -p tcp PORTA</code>	Sim (banco de política)
Isolar processo	<code>semanage permissive</code>	<code>semanage permissive -a DOMÍNIO</code>	Sim

✅ Checklist Final

SELinux Modo

- `setenforce 1` (runtime) + `SELINUX=enforcing` no config (persistente)
- `setenforce 0` (runtime) + `SELINUX=permissive` no config (persistente)
- `getenforce` e `sestatus` para verificar o modo atual

Contexto de Arquivos

- `semanage fcontext -a -t TIPO 'REGEX'` — regra no banco de políticas
- `restorecon -Rv /dir` — aplicar regra ao disco (SEMPRE em par)
- `ls -Z` para ver o contexto SELinux de arquivos

Booleans

- `setsebool -P BOOLEAN 1` (`-P` obrigatório para persistência; output: `BOOL --> on`)
- `getsebool BOOLEAN` para verificar estado
- `semanage boolean -l | grep TERMO` para encontrar o boolean correto

Porta Não-Padrão

- Apache porta 82: `httpd.conf` (`Listen 82`) + `semanage port` + `firewall-cmd`
- `semanage port -a -t http_port_t -p tcp 82`
- `man semanage-port` → seção EXAMPLES para consulta no exame

Domínio Permissivo

- `semanage permissive -a httpd_t` — isola apenas aquele processo
- Sistema continua em Enforcing para todos os outros processos

Firewall

- `firewall-cmd --permanent --add-service={http,https} && --reload`
- `firewall-cmd --permanent --add-port=82/tcp && --reload`
- `--permanent` sem `--reload` : salvo, não ativo. `--reload` sem `--permanent` : perde no reboot

Rich Rules

- Rich rule com `source address` para filtrar por IP de origem

☐ Remover serviço genérico ANTES de adicionar rich rule restritiva

☐ `accept` para rede autorizada ANTES do `reject` geral

RHCSA EX200 — Resumo Consolidado

Troubleshooting · Filesystems · Boot Recovery — com 1 exemplo de exame por tópico

Domínios: Operate Running Systems · Create and Configure File Systems | Baseado no Complemento (Resumo 8), no modelo de exemplos do Resumo 2

Bloco	Conteúdo	Ferramenta principal
1	Verificar/reparar filesystem	<code>fsck / xfs_repair</code>
2	Recuperar /etc/fstab corrompido	<code>rescue/emergency + mount -a</code>
3	Serviço systemd que não sobe	<code>systemctl + journalctl</code>
4	Recuperar senha root / boot crítico	GRUB + <code>rescue/emergency target</code>
5	Processo consumindo recursos	<code>top / ps / pgrep</code>
6	Outros cenários (SELinux, disco cheio, login, rede)	múltiplas ferramentas
7	Armadilhas e checklist final	referência rápida

BLOCO 1 · Filesystem corrompido — fsck (ext4) / xfs_repair (XFS)

REGRA DE OURO: nunca rode `fsck/xfs_repair` em filesystem **montado** — sempre `umount` antes.

Filesystem	Verificar (dry-run)	Reparar
ext4	<code>fsck -n /dev/sdX1</code>	<code>fsck -y /dev/sdX1</code>
XFS	<code>xfs_repair -n /dev/sdX1</code>	<code>xfs_repair /dev/sdX1</code>

Antes de reparar, descubra o tipo: `lsblk -f` ou `blkid /dev/sdX1`.

Exemplo completo (S2-Q9): `/dev/sdb1` (XFS) não monta mais, erro de corrupção. Repare sem perder o disco.

```
blkid /dev/sdb1           # confirmar TYPE="xfs"
umount /dev/sdb1          # OBRIGATORIO antes de reparar
xfs_repair -n /dev/sdb1   # dry-run: so mostra os problemas
xfs_repair /dev/sdb1      # repara de fato
mount /dev/sdb1 /mnt/dados # remontar
df -h /mnt/dados          # confirmar
```

Se `umount` falhar com "target is busy": `fuser -m /dev/sdb1` para achar o processo, `fuser -km /dev/sdb1` para matá-lo.

BLOCO 2 · /etc/fstab corrompido — sistema não sobe

REGRA MAIS IMPORTANTE DA PROVA: depois de editar `/etc/fstab`, rode `mount -a` antes de reiniciar. Erro no `fstab` trava o boot.

Exemplo completo (S4-Q3): você editou o `fstab` com um UUID errado e, ao reiniciar, o boot trava em "A start job is running for `/dev/sdb1`".

```
# 1. No GRUB: pressione 'e', va ate a linha "linux", adicione no final:
systemd.unit=emergency.target      # Ctrl+X para bootar

# 2. Remontar / como leitura/escrita (emergency monta ro por padrao)
mount -o remount,rw /

# 3. Corrigir a linha errada
vim /etc/fstab

# 4. TESTAR antes de reiniciar - nunca pule este passo
mount -a                          # sem erro = fstab esta correto

# 5. Reiniciar normalmente
systemctl reboot
```

Alternativa de verificação sem montar nada: `findmnt --verify`.

BLOCO 3 · Serviço systemd não inicia

Fluxo fixo: **status** → **logs** → **tentar iniciar** → **causa** → **corrigir** → **verificar**.

Causa	Como diagnosticar
Porta em uso	<code>ss -tlnp grep :80</code>
Config inválida	<code>httpd -t</code> (Syntax OK?)

SELinux	ausearch -m avc -ts recent / ls -Z
Unit file quebrado	systemctl cat NOME + systemctl daemon-reload

Exemplo completo (S6-Q12): httpd falha ao iniciar; suspeita é porta ocupada.

```
systemctl status httpd -l          # -l = log completo, sem truncar
journalctl -u httpd -b             # logs do boot atual
ss -tlnp | grep :80               # quem esta usando a porta 80?
# saida mostra outro processo (ex: nginx) na porta 80
systemctl stop nginx              # liberar a porta
systemctl start httpd
systemctl enable --now httpd      # garantir que sobe no boot
systemctl is-active httpd         # deve retornar "active"
```

BLOCO 4 · Rescue / Emergency target — boot crítico e senha root

	rescue.target	emergency.target
Quando usar	Serviço, fstab	Filesystem corrompido, boot crítico
Senha root	Exige	Geralmente não exige
Filesystems montados	Todos	Só / (read-only)
Acesso via GRUB	systemd.unit=rescue.target	systemd.unit=emergency.target

Exemplo completo (S1-Q22): você esqueceu a senha de root e precisa redefini-la via modo de recuperação.

```
# 1. No GRUB: 'e' na entrada do kernel -> ir ate a linha "linux" -> adicionar no final:
systemd.unit=rescue.target          # Ctrl+X para bootar

# 2. Sempre a primeira acao: remontar / como read-write
mount -o remount,rw /

# 3. Redefinir a senha
passwd root

# 4. Voltar ao boot normal
systemctl reboot
```

Via systemctl (sistema já rodando, sem reboot): systemctl isolate rescue.target / systemctl default para voltar.

BLOCO 5 · Monitoramento de processos e recursos

Ferramenta	Uso típico
top -b -n 1	snapshot não-interativo de CPU/mem
ps aux --sort=-%cpu	processos ordenados por CPU
pgrep -a NOME / pkill -9 NOME	achar/matar por nome
ss -tlnp	quem escuta em cada porta
free -h / df -h / du -sh	memória / disco

Exemplo completo (S3-Q18): um processo está consumindo CPU excessiva e travando o sistema. Identifique e encerre.

```
top -b -n 1 | head -15           # snapshot rapido, ordenado por CPU
ps aux --sort=-%cpu | head -5    # confirma o PID e o comando
pgrep -a stress                  # localizar PIDs pelo nome do processo
kill -9 <PID>                    # encerrar forçado
# ou, se houver varios com o mesmo nome:
pkill -9 stress
ps aux | grep stress             # confirmar que nao ha mais processos
```

BLOCO 6 · Outros cenários clássicos (1 exemplo de cada)

6.1 — Serviço web com erro 403 (SELinux)

Exemplo (S5-Q7): Apache sobe, mas retorna 403 ao acessar a página.

```
systemctl status httpd -l
ls -Z /var/www/html/          # contexto SELinux dos arquivos
ausearch -m avc -ts recent     # negacoes recentes
restorecon -Rv /var/www/html/  # corrigir contexto
ls -la /var/www/html/          # conferir permissoes tambem
systemctl restart httpd
```

6.2 — Disco cheio

Exemplo (S2-Q14): /var está com 100% de uso.

```
df -h # qual filesystem esta cheio
du -sh /var/log/ /var/cache/ # o que ocupa mais espaco
find /var -size +100M -type f 2>/dev/null # arquivos grandes
journalctl --vacuum-size=100M # reduzir o journal
find /var/log -name '*.gz' -delete # limpar logs antigos comprimidos
df -h # confirmar espaco liberado
```

6.3 — Usuário não consegue logar

Exemplo (S4-Q20): login de "carla" falha mesmo com senha correta.

```
id carla # existe? grupos ok?
chage -l carla # conta/senha expirada?
grep carla /etc/shadow # "!" antes do hash = bloqueada
usermod -U carla # desbloquear
usermod --expiredate '' carla # remover expiracao
grep carla /etc/passwd # shell existe? (which bash)
```

6.4 — Rede não funciona

Exemplo (S6-Q5): interface eth0 não pega IP após reboot.

```
ip addr show # interface tem IP?
nmcli dev status # estado da conexao
nmcli con show # perfis existentes
nmcli con up NOME_DA_CONEXAO # ativar o perfil
ping -c2 8.8.8.8 # testar conectividade externa
```

BLOCO 7 · Armadilhas Críticas

Certo	Errado
umount sempre antes de fsck/xfs_repair	rodar em filesystem montado (corrompe)
mount -a logo após editar /etc/fstab	reiniciar sem testar o fstab
mount -o remount,rw / antes de editar qualquer arquivo no rescue/emergency	tentar editar com / ainda ro
xfs_repair para XFS, fsck para ext4	usar a ferramenta errada para o tipo de FS
ss -tlnp grep :PORTA para achar conflito de porta	assumir que é sempre erro de config
usermod -U para desbloquear conta com "!" no shadow	mexer direto no /etc/shadow

Checklist Final

Filesystem

- ☐ Sempre umount antes de fsck/xfs_repair
- ☐ Sei identificar o tipo de FS com lsblk -f / blkid antes de escolher a ferramenta

Fstab / Boot

- ☐ mount -a depois de editar /etc/fstab, sempre, antes de reiniciar
- ☐ Sei entrar em emergency.target / rescue.target via GRUB (systemd.unit=...)
- ☐ Sei que / fica ro no emergency e preciso de mount -o remount,rw /

Serviços

- ☐ Fluxo: systemctl status -l → journalctl -u -b → corrigir → systemctl start
- ☐ Sei checar porta ocupada (ss -tlnp) e SELinux (ausearch -m avc -ts recent)

Recuperação crítica

- ☐ Sei redefinir senha de root via rescue target

Monitoramento

- ☐ ps aux --sort=-%cpu / top -b -n 1 / pgrep / pkill de cabeça

Outros cenários

- ☐ Disco cheio: df -h + du -sh + find -size +100M
- ☐ Login: chage -l + grep usuario /etc/shadow + usermod -U
- ☐ Rede: ip addr show + nmcli con up

RHCSA EX200 — Resumo Dia 08/05

SSH · find · tar · Autofs/NFS · Flatpak · Boot/GRUB · Hostname · SCP

PROVA: 14/05

! Armadilhas do Dia

Certo	Errado
Testar login por chave ANTES de desabilitar PasswordAuthentication	Desabilitar PasswordAuthentication antes de testar — pode travar o acesso (lock-out)
tar -czf (gzip usa -z) · tar -cjf (bzip2 usa -j)	Confundir -z com -j — gera arquivo no formato errado
--exclude='*.pdf' ANTES da fonte, entre aspas simples	--exclude depois da fonte, ou sem aspas — o shell expande o glob antes
-mtime -30 = modificado há MENOS de 30 dias (recente)	Interpretar -30 como 'mais antigo que 30 dias' — o sinal é invertido
fstab NFS sempre com a opção _netdev	Esquecer _netdev — o boot pode travar esperando a rede subir
hostnamectl set-hostname — persiste (grava /etc/hostname)	hostname NOME — muda só em memória, não persiste no reboot
systemctl set-default = permanente (próximos boots)	systemctl isolate = confundir com permanente — isolate só vale para o boot atual

Tabela-Chave — Mapa de Blocos por Simulado

Bloco	Conteúdo	Simulados
1	SSH — chaves, hardening, sshd_config	S1-Q16 · S2-Q19 · S3-Q17 · S4-Q15 · S5-Q19 · S6-Q23
2	find — -size, -mtime, -user, -exec, -perm	S1-Q9 · S2-Q12 · S3-Q14 · S4-Q28 · S5-Q12 · S6-Q14
3	tar — gzip, bzip2, exclusões, -C	S1-Q14 · S2-Q17 · S4-Q14 · S5-Q17 · S5-Q23
4	Autofs / NFS — auto.master + auto.data	S1-Q18 · S2-Q22
5	Flatpak — remote-add + install	S1-Q5 · S2-Q27 · S3-Q9 · S4-Q17 · S5-Q10 · S6-Q15
6	Boot / GRUB / Hostname / /etc/hosts	S1-Q10 · S2-Q10 · S2-Q11 · S4-Q24 · S5-Q20 · S6-Q30
7	Boot Targets — multi-user vs graphical	S1-Q26 · S4-Q7 · S5-Q7 · S6-Q3 · S6-Q7
8	SCP / rsync — transferência segura	S1-Q29

BLOCO 1 — SSH: Chaves e Hardening

A autenticação por chave é mais segura que senha. O fluxo sempre segue a mesma ordem: (1) gerar par de chaves → (2) copiar chave pública → (3) testar → (4) hardening no sshd_config. Nunca desabilite PasswordAuthentication antes de confirmar que o login por chave funciona.

Fluxo completo de referência rápida:

```
# 1. GERAR par de chaves (executar como o usuário que vai usar)
ssh-keygen -t rsa # Enter em todos os prompts (sem passphrase no exame)
# Gera: ~/.ssh/id_rsa (privada) e ~/.ssh/id_rsa.pub (pública)

# 2. COPIAR chave pública para o servidor (pede senha uma única vez)
ssh-copy-id user@servidor
ssh-copy-id root@localhost # testar localmente

# 3. TESTAR login sem senha
ssh user@servidor # deve entrar direto
```

```
# 4. HARDENING – editar /etc/ssh/sshd_config
PasswordAuthentication no          # desabilitar autenticação por senha
PermitRootLogin no                 # bloquear root via SSH (ou yes, se o exame pedir)

# 5. VERIFICAR antes de reiniciar (evita lock-out)
sshd -T | grep -i passwordauthentication
systemctl restart sshd
```

■ SEMPRE teste o login por chave ANTES de desabilitar PasswordAuthentication. Se travar o acesso no exame, você perde pontos sem poder corrigir.

Exemplo completo (S1-Q16): chave RSA para root + desabilitar senha

```
ssh-keygen -t rsa                  # [Enter] em todos os prompts
ssh-copy-id root@localhost         # digitar a senha do root quando pedir
ssh localhost                      # testar – deve entrar sem senha
exit

vim /etc/ssh/sshd_config
# PasswordAuthentication no
sshd -T | grep -i passwordauthentication # confirmar: ...no
systemctl restart sshd
```

Exemplo completo (S2-Q19): criar usuário alice + chaves + hardening duplo

alice precisa ter senha para que o ssh-copy-id consiga autenticar e instalar a chave pela primeira vez.

```
useradd alice
passwd alice                        # senha temporária

ssh-keygen -t rsa
ssh-copy-id alice@serverb          # ou IP se hostname não resolve

ssh alice@serverb                  # testar ANTES de desabilitar senha

vim /etc/ssh/sshd_config
# PermitRootLogin no
# PasswordAuthentication no
systemctl restart sshd
```

Exemplo completo (S3-Q17): padrão invertido — permitir root via SSH

Este simulado pede o oposto do padrão: habilitar o root login em vez de bloquear.

```
vim /etc/ssh/sshd_config
# PermitRootLogin yes
systemctl restart sshd

ssh-keygen -t rsa
ssh-copy-id root@192.168.1.4       # usar IP é mais confiável
ssh root@192.168.1.4              # deve entrar sem senha
```

■ Usar IP em vez de hostname é mais seguro no exame — evita falhas de resolução DNS/hosts que podem impedir o ssh-copy-id de funcionar.

Variação (S5-Q19 / S6-Q23): gerar as chaves como o próprio usuário (não como root)

```
useradd john
passwd john
su - john                          # trocar para john ANTES de gerar as chaves
ssh-keygen -t rsa
ssh-copy-id john@localhost
exit                                # voltar para root

vim /etc/ssh/sshd_config
# PasswordAuthentication no
systemctl restart sshd
ssh john@localhost                 # sem senha
```

■ Fazer o ssh-keygen como o próprio usuário garante que as chaves fiquem no diretório correto (~john/.ssh/). Se fizer como root, a chave fica em /root/.ssh/ — não funciona para o login de john.

BLOCO 2 — find: Filtros Avançados

O find percorre a árvore de diretórios aplicando filtros. Os filtros são AND por padrão (todos precisam ser verdadeiros). A ação `-exec` executa um comando em cada resultado — `{}` é substituído pelo caminho encontrado, e `\;` termina o `-exec`.

Filtro	Significado	Exemplo
<code>-size +3M</code>	Maior que 3 MiB	<code>find /etc -size +3M</code>
<code>-size -5M</code>	Menor que 5 MiB	<code>find /etc -size -5M</code>
<code>-mtime -30</code>	Modificado nos últimos 30 dias	<code>find /etc -mtime -30</code>
<code>-mtime +30</code>	Modificado há mais de 30 dias	<code>find /etc -mtime +30</code>
<code>-type f / -type d</code>	Arquivo regular / diretório	<code>find /usr -type f</code>
<code>-user sam</code>	Pertence ao usuário sam	<code>find /home -user sam</code>
<code>-name '*.conf'</code>	Nome termina em .conf	<code>find /etc -name '*.conf'</code>
<code>-perm -4000</code>	SUID ativo	<code>find /usr -perm -4000</code>

Exemplo completo (S1-Q9): arquivos em /etc > 3MiB → copiar sem sobrescrever

```
mkdir -p /find/largefiles
find /etc -size +3M -exec cp -n '{}' /find/largefiles/ \;
ls /find/largefiles/
```

■ `cp -n` = no-clobber (não sobrescreve). `cp -u` = só copia se a fonte for mais recente. O exame especifica qual usar — leia o enunciado.

Exemplo completo (S2-Q12 / S6-Q14): arquivos de root → copiar preservando o filtro -user

```
mkdir -p /find/rootfiles
find /usr/bin -type f -user root -exec cp {} /find/rootfiles/ \;
ls /find/rootfiles/ | head
```

Exemplo completo (S5-Q12): múltiplos filtros combinados + chmod nos resultados

`-type f`: só arquivos. `-size +5M`: maiores que 5 MiB. `-mtime -30`: modificados há MENOS de 30 dias (recente). `-name '*.conf'`: aspas simples evitam expansão do shell.

```
mkdir -p /find/recent_configs
find /etc -type f -size +5M -mtime -30 -name '*.conf' \
    -exec cp {} /find/recent_configs/ \;

chmod 600 /find/recent_configs/*
ls -l /find/recent_configs/
```

■ `-mtime -30` = modificado há MENOS de 30 dias (recente). `-mtime +30` = modificado há MAIS de 30 dias (antigo). O sinal é invertido em relação ao que parece intuitivo.

Variações do padrão 'copiar preservando atributos' (S4-Q28):

```
mkdir -p /find/sam_files
# cp -rp: recursivo + preserva permissões/dono/timestamps
find /home -user sam -exec cp -rp {} /find/sam_files/ \;
# OU cp -a (archive – ainda mais completo, preserva SELinux contexts)
find /home -user sam -exec cp -a {} /find/sam_files/ \;
```

BLOCO 3 — tar: Criar, Listar e Extrair

O tar empacota e (opcionalmente) comprime arquivos. A flag `-f` deve vir imediatamente antes do nome do arquivo. Ordem mais comum: `tar -czf SAIDA.tar.gz FONTE`.

Flag	Significado	Extensão comum
-c	Create (criar)	.tar.gz / .tar.bz2
-x	eXtract (extrair)	.tar.gz / .tar.bz2
-t	lisT (listar sem extrair)	.tar.gz
-f	File (nome do arquivo)	sempre o último antes do nome
-z	gZip compression	.tar.gz ou .tgz
-j	bzip2 compression	.tar.bz2 ou .tbz2
-J	xz compression	.tar.xz
-C	Change dir (destino)	tar -xzf arq.tar.gz -C /dir
--exclude	Excluir padrão	--exclude='*.pdf'

■ Mnemônico: -z = gZip, -j = bzip2 (bzip2 é 'mais avançado' que gzip), -J = xZ (maiúsculo).

Exemplo completo (S1-Q14): gzip + listar sem extrair

```
tar -czf /root/config_backup.tar.gz /etc/ssh
tar -tf /root/config_backup.tar.gz      # listar SEM extrair
```

Exemplo completo (S2-Q17): bzip2 + extrair para diretório específico

```
mkdir -p /archive/restored/myetc
tar -cjf /archive/myetc.tbz2 /etc      # -j = bzip2
tar -xjf /archive/myetc.tbz2 -C /restored/myetc/  # -C muda o destino
ls /restored/myetc/
```

Exemplo completo (S4-Q14): múltiplas fontes + restaurar

```
mkdir -p /archive/restored
tar -czf /archive/sysconfig.tar.gz /etc /root/anaconda-ks.cfg
tar -tf /archive/sysconfig.tar.gz | head -5

tar -xzf /archive/sysconfig.tar.gz -C /restored
```

Exemplo completo (S5-Q17): gzip excluindo um padrão de arquivo

```
mkdir -p /Backup
tar -czf /Backup/myhome.tgz --exclude='*.pdf' /home
tar -tf /Backup/myhome.tgz | grep '\.pdf'  # não deve retornar nada
```

■ --exclude vem ANTES do caminho de origem. As aspas simples em '*.pdf' são obrigatórias — sem elas, o shell expande o glob antes de passar para o tar.

BLOCO 4 — Autofs / NFS: Montagem sob Demanda

O Autofs monta o compartilhamento NFS apenas quando é acessado e desmonta automaticamente quando inativo — mais eficiente que /etc/fstab para NFS. Requer dois arquivos: auto.master (diretório base + mapa) e um arquivo de mapa.

■ Diferença crítica: fstab NFS = sempre montado (estático). Autofs = monta sob demanda (dinâmico). O exame especifica qual usar.

Exemplo completo (S1-Q18): Autofs para 192.168.1.100:/shares/public

```
dnf install nfs-utils autofs -y

vim /etc/auto.master
# Adicionar: /data /etc/auto.data      (formato: /BASEDIR /ARQUIVO_MAPA)

vim /etc/auto.data
# Conteúdo: public -rw, sync 192.168.1.100:/shares/public
# (formato: SUBDIR OPCOES SERVIDOR:/CAMINHO)

systemctl enable --now autofs
```

```
# TESTAR – o simples acesso dispara a montagem
cd /data/public
ls      # deve mostrar o conteúdo do compartilhamento NFS
```

■ O Autofs cria /data automaticamente — NÃO precisa fazer mkdir. cd /data/public é o que dispara a montagem; sem acesso, não há montagem (economia de recursos).

Exemplo completo (S2-Q22): NFS via fstab (não autofs) com _netdev

```
mkdir /nfs_share                # com fstab o diretório precisa existir antes

vim /etc/fstab
# Adicionar: servera:/share /nfs_share nfs defaults,_netdev 0 0

mount -a                        # montar sem reboot
df -h /nfs_share                # confirmar
```

■ _netdev é OBRIGATÓRIO para NFS no fstab. Sem ele, o sistema pode travar no boot tentando montar o NFS antes da rede estar disponível.

BLOCO 5 — Flatpak: Repositório e Instalação

O Flatpak é o novo padrão RHEL 10 para apps desktop, isolando a aplicação em sandbox. O comando remote-add é equivalente ao arquivo .repo do DNF. --if-not-exists evita erro se o repositório já foi adicionado antes.

Gabarito padrão — todos os simulados seguem a mesma estrutura de 3 comandos:

```
# PASSO 1: adicionar Flathub (uma vez só)
flatpak remote-add --if-not-exists flathub \
  https://dl.flathub.org/repo/flathub.flatpakrepo

# PASSO 2: instalar o app pedido (trocar o ID conforme a questão)
flatpak install flathub [ID_DO_APP] -y

# PASSO 3: verificar
flatpak list
```

Aplicativos já cobrados:

Simulado	Aplicativo	ID Flatpak
S1-Q5	GNOME Text Editor	org.gnome.TextEditor
S2-Q27	GIMP	org.gimp.GIMP
S3-Q9	GNOME Calculator	org.gnome.Calculator
S4-Q17	KCalc (KDE Calculator)	org.kde.kcalc
S5-Q10	Mozilla Firefox	org.mozilla.firefox
S6-Q15	GNOME Calculator	org.gnome.Calculator

■ --if-not-exists é fundamental no exame. Se você já executou o remote-add antes (em outra questão do mesmo simulado), sem essa flag o comando daria erro.

BLOCO 6 — Boot / GRUB / Hostname / /etc/hosts

Exemplo completo (S1-Q10): remover rhgb e quiet do kernel (persistente)

grubby é o método preferido no RHEL — não exige editar o grub.cfg diretamente. --update-kernel=ALL aplica a mudança a todos os kernels instalados.

```
grubby --update-kernel=ALL --remove-args='rhgb quiet'
grubby --info=DEFAULT | grep args      # rhgb e quiet não devem aparecer
```

■ *grubby é mais seguro que editar /boot/grub2/grub.cfg diretamente — erros no grub.cfg podem impedir o boot.*

Exemplo (S2-Q10): instalar kernel mais recente e confirmar como padrão

```
dnf update kernel -y          # nunca remove o kernel anterior
grubby --default-kernel       # deve mostrar o kernel mais novo
reboot
uname -r                      # confirmar versão nova
```

Exemplo (S2-Q11): hostname estático rhel.server.com

```
hostnamectl set-hostname rhel.server.com
hostnamectl
cat /etc/hostname             # deve mostrar rhel.server.com
```

■ *hostnamectl set-hostname atualiza /etc/hostname automaticamente. O comando 'hostname' sozinho só muda em memória — não persiste.*

Exemplo completo (S5-Q20): hostname + /etc/hosts para resolução local e externa

```
hostnamectl set-hostname rhel.server.com

vim /etc/hosts
# Garantir estas linhas:
127.0.0.1    localhost localhost.localdomain rhel.server.com
192.168.1.6  rhel.server.com

hostname
ping -c1 rhel.server.com
```

Exemplo (S4-Q24): boot target padrão como rescue.target

```
systemctl set-default rescue.target
systemctl get-default           # rescue.target
```

■ *rescue.target exige senha de root para acessar o shell. Depois do exame, lembre de voltar para multi-user.target se necessário.*

Exemplo completo (S6-Q30): grubby — listar índices, definir índice 1, remover quiet

Índice 0 = kernel mais novo. Índice 1 = kernel anterior. Definir índice 1 como padrão simula um rollback.

```
grubby --info=ALL | grep -E '^(index|kernel) '

grubby --set-default-index=1
grubby --update-kernel=ALL --remove-args='quiet'
grubby --info=DEFAULT
```

BLOCO 7 — Boot Targets: multi-user vs graphical

Os boot targets do systemd substituem os runlevels do SysV. set-default muda o target permanentemente (próximos reboots). isolate muda o target AGORA, sem reboot.

Target	Equivalente	Quando usar
multi-user.target	Runlevel 3	Servidor sem GUI (economiza RAM/CPU)
graphical.target	Runlevel 5	Desktop com GUI (GNOME instalado)
rescue.target	Runlevel 1	Manutenção (requer senha root)
emergency.target	Mínimo	Recuperação crítica (só root fs)

Exemplo completo (S1-Q26 / S5-Q7 / S6-Q3): multi-user.target

```
systemctl set-default multi-user.target
systemctl get-default           # multi-user.target
```

Exemplo completo (S4-Q7 / S6-Q7): graphical.target

```
systemctl set-default graphical.target
systemctl get-default           # graphical.target
```

■ *set-default: permanente (próximos boots). isolate: temporário (só o boot atual). No exame, use set-default, pois a questão sempre pede persistência.*

BLOCO 8 — SCP / rsync: Transferência Segura

O scp transfere arquivos usando SSH. O rsync é mais eficiente para grandes volumes: só transfere as diferenças e retoma transferências interrompidas. Ambos são aceitos no exame.

Exemplo completo (S1-Q29): copiar arquivo preservando atributos

```
# Opção 1: scp -p (preserva timestamps e permissões)
scp -p /root/anaconda-ks.cfg root@ServerB:/tmp/

# Opção 2: rsync -a (preserva TUDO – dono, permissões, timestamps, links, SELinux)
rsync -av /root/anaconda-ks.cfg root@ServerB:/tmp/

ls -la /tmp/anaconda-ks.cfg
```

Característica	scp	rsync
Preserva atributos	Com -p	Com -a (mais completo)
Transferência parcial	Reinicia do zero	Retoma de onde parou
Sincronia	Sempre copia tudo	Só copia diferenças
Links simbólicos	Não (segue o link)	Sim (com -a)

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
SSH — ordem de hardening	Testar chave → PasswordAuthentication no	PasswordAuthentication no → testar depois
tar gzip vs bzip2	gzip = -z / bzip2 = -j	tar -czf arq.tbz2 (flag errada)
tar --exclude posição	--exclude='*.pdf' ANTES da fonte	--exclude depois da fonte
find -mtime -30	Menos de 30 dias (recente)	Interpretar como mais antigo
Autofs — diretório base	Autofs cria /data automaticamente	mkdir /data antes (desnecessário)
fstab NFS	Sempre usar _netdev	Sem _netdev — boot pode travar
set-default vs isolate	isolate = agora / set-default = permanente	isolate muda permanentemente
hostname vs hostnamectl	hostnamectl = persiste (grava /etc/hostname)	hostname = só em memória, não persiste

✓ Checklist Final

SSH

- ☐ ssh-keygen -t rsa → ssh-copy-id user@host → ssh user@host (sem senha)
- ☐ PasswordAuthentication no + sshd -T para verificar + systemctl restart sshd
- ☐ PermitRootLogin yes/no conforme o exame pedir — leia o enunciado

find

- ☐ find -size +3M -exec cp -n {} /dest/ \; (copiar sem sobrescrever)
- ☐ find -type f -user root -exec cp {} /dest/ \;

- ☐ `find -size +5M -mtime -30 -name '*.conf'` (múltiplos filtros — todos com AND)
- ☐ `-mtime -30` = RECENTE (menos de 30 dias). `-mtime +30` = ANTIGO (mais de 30 dias)

tar

- ☐ `tar -czf arq.tar.gz /fonte` (criar gzip)
- ☐ `tar -cjf arq.tbz2 /fonte` (criar bzip2) — `-j` não é `-z`
- ☐ `tar -tf arq.tar.gz` (listar sem extrair)
- ☐ `tar -xzf arq.tar.gz -C /destino` (extrair para diretório específico)
- ☐ `tar --exclude='*.pdf' /fonte` (aspas simples obrigatórias, antes da fonte)

Autofs / Flatpak

- ☐ `auto.master: /data /etc/auto.data + auto.data: public -rw SERVER:/path`
- ☐ `fstab NFS com _netdev: servera:/share /mnt nfs defaults,_netdev 0 0`
- ☐ `flatpak remote-add --if-not-exists flathub URL + flatpak install flathub APP -y`

Boot / GRUB / Hostname

- ☐ `grubby --update-kernel=ALL --remove-args='rhgb quiet'`
- ☐ `grubby --set-default-index=1` (rollback para kernel anterior)
- ☐ `hostnamectl set-hostname` + verificar com 'hostname' e `cat /etc/hostname`
- ☐ `/etc/hosts`: mapear hostname para 127.0.0.1 E para o IP externo
- ☐ `systemctl set-default multi-user.target / graphical.target + get-default`

SCP / rsync

- ☐ `scp -p arquivo user@host:/dest` (preservar atributos)
- ☐ `rsync -av arquivo user@host:/dest` (archive — mais completo que `scp -p`)

RHCSA EX200 — Cheatsheet Completo

Red Hat Certified System Administrator · RHEL 10 · Prova: 14/05

1 · RESET DE SENHA ROOT (aparece em TODOS os simulados)

Fluxo obrigatório: Reiniciar → GRUB (e) → editar linha linux → Ctrl+X

```
# Na linha que começa com 'linux':  
# 1. Trocar 'ro' por 'rw'  
# 2. Adicionar no final: init=/bin/bash  
  
passwd root # digitar nova senha  
touch /.autorelabel # relabel SELinux (OBRIGATÓRIO)  
exec /sbin/init # reiniciar corretamente
```

■ Sem touch /.autorelabel → SELinux impede login. Sem rw → /etc/shadow é read-only.

2 · REPOSITÓRIO DNF / YUM

Arquivo: /etc/yum.repos.d/nome.repo

```
[BaseOS]  
name=RHEL 10 BaseOS  
baseurl=file:///mnt/BaseOS # ou http://IP/path/BaseOS  
enabled=1  
gpgcheck=0 # ou gpgcheck=1 + gpgkey=file:///etc/pki/rpm-gpg/...  
  
[AppStream]  
name=RHEL 10 AppStream  
baseurl=file:///mnt/AppStream  
enabled=1  
gpgcheck=0
```

ISO no fstab (persistente):

```
# /etc/fstab  
/path/RHEL-10.iso /mnt iso9660 loop,defaults 0 0  
mount -a && dnf clean all && dnf repolist
```

3 · REDE — nmcli

```
# Criar perfil com IPv4 + IPv6  
nmcli con add con-name lab-link ifname enp0s3 type ethernet \  
ipv4.method manual ipv4.addresses 192.168.1.10/24 \  
ipv4.gateway 192.168.1.1 ipv4.dns 8.8.8.8 \  
ipv6.method manual ipv6.addresses fd00::10/64 \  
ipv6.gateway fd00::1 connection.autoconnect yes  
  
# ADICIONAR IP secundário (+ é obrigatório para não sobrescrever)  
nmcli con mod lab-link +ipv4.addresses 10.0.0.5/24  
nmcli con mod lab-link +ipv6.addresses fd01::200/64  
  
nmcli con up lab-link  
ip addr show enp0s3
```

■ Sem o + → sobrescreve o IP primário. Sempre use +ipv4.addresses para IPs adicionais.

Outros comandos úteis:

nmcli con show	listar conexões
nmcli con del NOME	deletar perfil
hostnamectl set-hostname nome.com	definir hostname persistente
timedatectl list-timezones grep America	listar timezones
timedatectl set-timezone America/Sao_Paulo	definir timezone

4 · LVM — Fluxo Completo

```
# Passo 1: Partição  
fdisk /dev/sdb # n → p → Enter → Enter → +2G → w  
partprobe /dev/sdb  
  
# Passo 2: Physical Volume  
pvcreate /dev/sdb1  
  
# Passo 3: Volume Group (PE size personalizado: -s 16M)  
vgcreate myvg /dev/sdb1 # padrão 4M  
vgcreate -s 16M myvg /dev/sdb1 # PE personalizado  
  
# Passo 4: Logical Volume  
lvcreate -n mylv -L 500M myvg # por tamanho
```

```
lvcreate -n mylv -l 50 myvg # por extents (50 * PE_size)

# Passo 5: Formatar e montar
mkfs.ext4 /dev/myvg/mylv
mkfs.xfs /dev/myvg/mylv
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a

# Passo 6: Expandir (filesystem automático com -r)
lvextend -r -L +500M /dev/myvg/mylv
lvextend -r -L 1G /dev/myvg/mylv # tamanho total

# Verificar
pvs && vgs && lvs
vgdisplay myvg | grep "PE Size"
lvdisplay /dev/myvg/mylv | grep "Current LE"
```

■ **-r no lvextend é obrigatório para redimensionar o filesystem junto. Sem -r → filesystem não cresce.**

4b · VDO & Thin Provisioning

```
# VDO (mínimo 5G físico no RHEL 10)
dnf install lvm2 vdo -y
pvcreate /dev/sdc && vgcreate vdovg /dev/sdc
lvcreate --type vdo --name myvdo --size 5G --virtualsize 50G vdovg
mkfs.xfs /dev/vdovg/myvdo
mkdir /vdo_data
echo "/dev/vdovg/myvdo /vdo_data xfs defaults 0 0" >> /etc/fstab
mount -a
vdstats --human-readable

# THIN PROVISIONING
lvcreate -L 500M -T thin_vg/my_thin_pool # criar pool
lvcreate -V 2G -T thin_vg/my_thin_pool -n thin_vol # criar volume thin
# OU em um comando só:
lvcreate --type thin --name thin_vol \
--thinpool my_thin_pool --size 500m --virtualsize 2G thin_vg

# Renomear LV
lvrename vg1/thinpool1 vg1/thinpool2
```

5 · SWAP

```
fdisk /dev/sdb # n → Enter → Enter → +512M → t → 82 → w
partprobe /dev/sdb
mkswap /dev/sdb2
lsblk -f /dev/sdb2 # pegar UUID

# /etc/fstab:
UUID=<uuid> none swap defaults 0 0

swapon -a && free -h
```

6 · SELinux

getenforce / sestatus	verificar modo atual
setenforce 1	enforcing agora (não persiste)
setenforce 0	permissive agora (não persiste)
vim /etc/selinux/config → SELINUX=enforcing	persistência após reboot
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"	adicionar regra de contexto
restorecon -Rv /web	aplicar regra ao disco (sempre após semanage)
semanage port -a -t http_port_t -p tcp 82	permitir porta não-padrão
semanage port -l grep http	listar portas http
setsebool -P httpd_use_nfs 1	boolean persistente (-P obrigatório)
getsebool httpd_use_nfs	verificar boolean
getsebool -a grep ftp	listar booleans ftp
semanage permissive -a httpd_t	modo permissive só para o httpd_t
ls -Z arquivo	ver contexto SELinux do arquivo

■ **semanage fcontext e restorecon SEMPRE em par. Um sem o outro não funciona.**

7 · Firewall — firewall-cmd

```
# Serviços
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service={http,https} # múltiplos

# Porta
firewall-cmd --permanent --add-port=82/tcp

# Rich rule (restringir por origem)
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" \
source address="192.168.1.0/24" service name="http" accept'

# SEMPRE recarregar após --permanent
firewall-cmd --reload
firewall-cmd --list-all
```

8 · Usuários, Grupos, Permissões

<code>useradd -u 1500 -s /sbin/nologin -e 2029-12-21 sam</code>	criar user com UID, sem shell, com expiração
<code>usermod -s /bin/bash sam</code>	alterar shell
<code>usermod -aG developers alex</code>	adicionar a grupo secundário
<code>groupadd developers</code>	criar grupo
<code>chage -l john</code>	ver políticas de senha do usuário
<code>id alex</code>	ver UID/GID do usuário
<code>grep sam /etc/passwd</code>	verificar entrada no passwd

Políticas globais:

```
touch /etc/skel/Welcome.txt # arquivo criado para todo novo user
vim /etc/login.defs → PASS_MAX_DAYS 90 # expiração de senha
vim /etc/security/pwquality.conf → minlen = 8 # tamanho mínimo
```

9 · Permissões Especiais · ACL · Umask

```
# SGID – herança de grupo em diretório
groupadd developers && mkdir /opt/dev-data
chown :developers /opt/dev-data
chmod 2770 /opt/dev-data # 2 = SGID, 77 = owner+group rwx, 0 = others nada

# Sticky bit – /tmp style (usuário só deleta o próprio)
chmod 1777 /data/shared # 1 = sticky, 777 = todos rwx

# ACL
setfacl -m u:alex:rw /arquivo # usuário alex: leitura+escrita
setfacl -m u:kevin:--- /arquivo # usuário kevin: sem acesso
setfacl -m g:finance:rx /groups/accounting # grupo finance: r+x
setfacl -m d:g:finance:rx /groups/accounting # default ACL (herança em novos arquivos)
getfacl /arquivo

# UMASK
# Cálculo: 666 - umask = permissão de arquivo | 777 - umask = diretório
# umask 027 → arquivo 640 (rw-r-----), dir 750 (rwxr-x---)
# umask 077 → arquivo 600 (rw-----), dir 700 (rwx-----)
echo "umask 027" >> /home/harry/.bashrc # persistente para harry
# Sistema todo: /etc/profile.d/custom.sh
```

■ Default ACL (d:) é obrigatório para novos arquivos herdarem permissão. Sem ela, só o diretório é afetado.

10 · SSH — Chaves e Hardening

```
ssh-keygen -t rsa # gerar par de chaves (Enter em tudo)
ssh-copy-id user@host # copiar chave pública para host remoto

# /etc/ssh/sshd_config
PasswordAuthentication no # desabilitar senha
PermitRootLogin no # bloquear root via SSH
PermitRootLogin yes # permitir (quando o exam pedir)

sshd -T | grep -i password # verificar config ativa antes de reiniciar
systemctl restart sshd

scp -p /root/file.txt root@server:/tmp/ # copiar com preservação de atributos
```

11 · Systemd Timers (novo no RHEL 10)

```
# /etc/systemd/system/myjob.service
[Unit]
Description=Meu Job
[Service]
Type=oneshot
ExecStart=/usr/local/bin/myscript.sh

# /etc/systemd/system/myjob.timer
[Unit]
Description=Timer Meu Job
[Timer]
OnCalendar=*-*-* 07:00:00 # todo dia às 07:00
OnCalendar=minutely # todo minuto
OnCalendar=daily # todo dia à meia-noite
OnCalendar=*-*-* 16:15:00 # 4:15 PM
OnBootSec=1m # 1 min após boot (para evitar timer preso)
OnUnitActiveSec=2h # repetir a cada 2h após última execução
Persistent=true # rodar se perder o horário
Unit=myjob.service
[Install]
WantedBy=timers.target

# Ativar
systemctl daemon-reload
systemctl enable --now myjob.timer
systemctl list-timers # verificar próxima execução
```

■ **SEMPRE** rodar **daemon-reload** antes de **enable --now**. Sem isso, **systemd** não vê os novos arquivos.

12 · Web Server — Apache (httpd)

```
dnf install httpd -y
echo "Hello RHCSA" > /var/www/html/index.html
systemctl enable --now httpd
firewall-cmd --permanent --add-service={http,https}
firewall-cmd --reload
curl http://localhost

# Porta não-padrão (ex: 82)
vim /etc/httpd/conf/httpd.conf → Listen 82
semanage port -a -t http_port_t -p tcp 82
firewall-cmd --permanent --add-port=82/tcp && firewall-cmd --reload
systemctl restart httpd
```

13 · Cron · at · Chrony · Tuned · Sysctl

```
# CRON (formato: Min Hora Dia Mês DiaSemana)
crontab -e -u root
45 0 * * * find /tmp -type f -empty -delete # 12:45 AM todo dia

# Controle de acesso ao cron
echo "tom" > /etc/cron.allow # só 'tom' pode usar cron (todos outros negados)

# AT (tarefa única)
at 23:30 # prompt 'at>' aparece
touch /var/tmp/check.txt # digitar comando
# Ctrl+D para salvar
atq # listar fila

# CHRONY
timedatectl set-timezone America/Sao_Paulo
dnf install chrony -y && systemctl enable --now chronyd
timedatectl set-ntp true
timedatectl # verificar NTP synchronized: yes

# TUNED
dnf install tuned -y && systemctl enable --now tuned
tuned-adm profile throughput-performance
tuned-adm profile virtual-guest powersave # combinar perfis
tuned-adm active

# SYSCTL (kernel params)
vim /etc/sysctl.d/99-forwarding.conf
# net.ipv4.ip_forward = 1
# net.ipv6.conf.all.forwarding = 1
sysctl -p /etc/sysctl.d/99-forwarding.conf
```

14 · find · tar · grep · sed

```
# FIND
find /etc -size +3M -exec cp -n '{}' /find/largefiles/ \;
find /etc -type f -size +5M -mtime -30 -name "*.conf" -exec cp {} /dest/ \;
find /usr/bin -type f -user root -exec cp {} /find/rootfiles/ \;
find /usr -type f -perm /4000 -size -10M # arquivos SUID < 10MB
find /tmp -type f -empty -delete # apagar arquivos vazios

# TAR
tar -czf /root/backup.tar.gz /etc/ssh # gzip
tar -cjf /archive/myetc.tbz2 /etc # bzip2
tar -czf /Backup/home.tgz --exclude='*.pdf' /home # excluindo .pdf
tar -tf arquivo.tar.gz # listar sem extrair
tar -xzf arquivo.tar.gz -C /destino/ # extrair para pasta

# GREP
grep -i "^Host" /etc/ssh/sshd_config > /root/hosts.txt # linhas que começam com Host
grep -ri "error" /var/log > /root/errors.txt # recursivo case-insensitive
grep "http" /etc/services > /root/http.txt

# SED (substituição)
sed 's/sam/Sam/g' letter > newsletter # substituir todas ocorrências
```

15 · Scripts Shell — Modelos Prontos

```
#!/bin/bash
# --- Loop for com lista ---
for USER in root adm ftp; do
  USERID=$(id -u $USER)
  echo "User $USER has ID $USERID"
done

# --- Loop com todos os argumentos (somar positivos) ---
TOTAL=0
for NUM in "$@"; do
  if [ "$NUM" -gt 0 ]; then TOTAL=$((TOTAL + NUM)); fi
done
echo "The sum is $TOTAL"

# --- While lendo arquivo linha a linha ---
while read DATE NAME; do
  echo "Name: $NAME was born on $DATE"
done < /root/dates.txt

# --- If/else com teste de arquivo ---
if [ -f "$1" ]; then echo "File exists."; else echo "File missing."; fi

# --- Case (case-insensitive com tr) ---
INPUT=$(echo "$1" | tr '[:upper:]' '[:lower:]')
case "$INPUT" in
  yes) echo "That is nice" ;;
  no) echo "I am sorry" ;;
  *) echo "Unknown argument" ;;
esac

# --- Read interativo ---
echo "Enter number:"; read NUM1

# --- Variáveis e export ---
export CLASS="RHCSA" # disponível para processos filhos
TODAY=$(date +%F) # capturar saída de comando
```

■ **chmod +x /usr/local/bin/script.sh** → obrigatório em todos os scripts!

16 · Flatpak · NFS · Autofs · Journald · Grubby

```
# FLATPAK
flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install flathub org.gnome.TextEditor -y
flatpak list

# NFS (fstab estático – não autofs)
# /etc/fstab:
servera:/share /nfs_share nfs defaults,_netdev 0 0
mkdir /nfs_share && mount -a

# AUTOFS (montagem sob demanda)
vim /etc/auto.master → /data /etc/auto.data
```

```

vim /etc/auto.data → public -rw, sync 192.168.1.100:/shares/public
systemctl enable --now autofs
cd /data/public # trigger da montagem

# JOURNALD PERSISTENTE
vim /etc/systemd/journald.conf
# Storage=persistent
# SystemMaxUse=100M
systemctl restart systemd-journald
journalctl --flush
journalctl -u sshd -b > /var/log/ssh_boot.log # filtrar por serviço + boot atual

# GRUBBY
grubby --update-kernel=ALL --remove-args="rhgb quiet"
grubby --set-default-index=1
grubby --default-kernel
grubby --info=ALL | grep index

# BOOTLOADER TIMEOUT (manual)
vim /etc/default/grub → GRUB_TIMEOUT=10
grub2-mkconfig -o /boot/grub2/grub.cfg

```

17 · Processos · Systemd · Boot Targets

<code>sleep 1000 &</code>	rodar em background
<code>jobs -l / pgrep -a sleep / ps aux grep sleep</code>	encontrar PID
<code>nice -n 15 sleep 1000 &</code>	iniciar com niceness 15
<code>renice -n 5 -p PID</code>	mudar niceness de processo rodando
<code>kill PID / kill -9 PID / kill -SIGINT PID</code>	SIGTERM / SIGKILL / SIGINT(2)
<code>ps -o pid,ni,comm -p PID</code>	ver PID, niceness e nome do processo
<code>systemctl enable --now httpd</code>	enable + start em um comando
<code>systemctl set-default multi-user.target</code>	boot em CLI (servidor)
<code>systemctl set-default graphical.target</code>	boot em GUI
<code>systemctl get-default</code>	ver target padrão
<code>systemctl set-default rescue.target</code>	boot em rescue mode

18 · Checklist de Verificação Pós-Tarefa

Tarefa	Comando de verificação
LVM montado	<code>df -h /ponto && lsblk</code>
Serviço ativo+enabled	<code>systemctl status NOME</code>
Firewall aplicado	<code>firewall-cmd --list-all</code>
SELinux enforcing	<code>getenforce</code>
Contexto SELinux	<code>ls -Z /arquivo</code>
Boolean SELinux	<code>getsebool NOME</code>
Repo funcionando	<code>dnf repolist</code>
Rede aplicada	<code>ip addr show enp0s3</code>
Timer ativo	<code>systemctl list-timers</code>
Swap ativo	<code>free -h && swapon -s</code>
ACL definida	<code>getfacl /arquivo</code>
Persistência reboot	<code>/etc/fstab + mount -a (sem erro)</code>
Script executável	<code>ls -l script.sh (x presente)</code>
Cron agendado	<code>crontab -l -u root</code>
NTP sincronizado	<code>timedatectl (NTP synchronized: yes)</code>

RHCSA EX200 — Revisão Geral (Dia 09/05)

Rsyslog · Alias/I/O Redirect · DNF Groups · Kernel · Troubleshooting de Permissões · 10 Fluxos Críticos · Simulado Mental

Véspera dos simulados cronometrados — 1 exemplo de exame por tópico, no modelo do Resumo 2 | Prova: 14/05

Bloco	Conteúdo	Questões de origem
1	Rsyslog — logging para arquivo específico	Sim5 Q30
2	Alias + I/O Redirect + personalização de shell	Sim2 Q21
3	DNF — grupos de pacotes e instalação via URL	Sim5 Q22, Sim2 Q29
4	Kernel update + grubby	Sim2 Q10, Sim5 Q6
5	Troubleshooting de permissões	Sim3 Q30, Sim4 Q30
6	10 fluxos mais críticos — revisão de velocidade	Todos os simulados
7	Simulado mental + plano de estudos + dicas finais	Auto-avaliação

BLOCO 1 · Rsyslog — logging para arquivo específico

Regra: `SELECTOR DESTINO`, onde selector = `FACILITY.PRIORITY`. Prioridades crescem: `debug < info < notice < warning < err < crit < alert < emerg`. Arquivo em `/etc/rsyslog.d/*.conf` é a forma mais limpa de customizar.

Exemplo completo (Sim5-Q30): enviar tudo com prioridade debug (e acima) para `/var/log/debug_messages.log`.

```
vim /etc/rsyslog.d/debug.conf
# conteudo (uma linha):
*.debug /var/log/debug_messages.log

systemctl restart rsyslog
systemctl status rsyslog           # confirmar que esta rodando

logger -p user.debug 'mensagem de teste' # gerar log de teste
cat /var/log/debug_messages.log        # confirmar que a msg chegou
```

CUIDADO: `*.debug` captura quase tudo. No exame use exatamente o selector pedido — não invente variações.

BLOCO 2 · Alias · I/O Redirect · Personalização de ambiente

Alias persistente vai em `~/.bashrc`. `source ~/.bashrc` aplica sem precisar reabrir sessão.

Exemplo completo (Sim2-Q21): alias 'myfiles' → 'ls -l /tmp' persistente para root; buscar 'http' em `/etc/services` sobrescrevendo `/root/http_services.txt`; depois acrescentar uma linha.

```
vim /root/.bashrc
# adicionar ao final:
alias myfiles='ls -l /tmp'

source /root/.bashrc           # aplicar sem logout
myfiles                        # testar

grep 'http' /etc/services > /root/http_services.txt # > SOBRESCREVE
echo 'Search Complete' >> /root/http_services.txt   # >> ACRESCENTA
tail -3 /root/http_services.txt                     # confirmar ultima linha
```

Operador	Ação	Se arquivo existe
>	Sobrescreve stdout	Apaga e recria
>>	Acrescenta stdout	Preserva
<	Redireciona entrada	Lê o arquivo
2> / 2>>	Sobrescreve/acrescenta stderr	Apaga / Preserva
&>	stdout + stderr juntos	Apaga e recria
2>&1	stderr para onde vai o stdout	Preserva destino

ATENÇÃO: "sobrescrever" = `>` | "acrescentar" = `>>`. Trocar os dois custa a questão inteira.

BLOCO 3 · DNF — Grupos de pacotes e instalação via URL

Exemplo completo (Sim5-Q22): buscar, inspecionar e instalar o grupo "Development Tools".

```
dnf group list hidden | grep -i development      # 'hidden' mostra TODOS os grupos
dnf group info "Development Tools"              # Mandatory / Optional / Default
dnf group install "Development Tools" -y        # aspas obrigatorias (nome com espaco)
dnf group list installed                        # confirmar
gcc --version && make --version                 # confirmar ferramentas
```

GRUPO MAIS COBRADO: "Development Tools" = gcc, make, autoconf, automake, binutils, glibc-devel, flex, bison.

Exemplo completo (Sim2·Q29): instalar o pacote zsh diretamente de uma URL, resolvendo dependências.

```
dnf install http://example.com/pub/zsh.rpm -y    # resolve dependencias automaticamente
which zsh && zsh --version
```

Característica	dnf install URL	rpm -ivh URL
Resolve dependências	SIM (automático)	NÃO (falha se faltar)
Uso no RHCSA	Preferido sempre	Apenas sem internet

BLOCO 4 · Kernel Update — dnf update kernel + grubby

dnf update kernel nunca remove o kernel anterior — apenas adiciona. grubby gerencia o kernel padrão de boot.

Exemplo completo (Sim2·Q10 / Sim5·Q6): instalar o kernel mais recente, garantir que é o padrão de boot e manter o anterior como fallback.

```
dnf update kernel -y                                # kernel anterior e MANTIDO

grubby --default-kernel                             # confirmar qual sera usado no boot
grubby --info=ALL | grep '^kernel'                  # index=0 (novo) / index=1 (anterior)

reboot
uname -r                                             # confirmar versao ativa apos reboot
grubby --info=ALL | grep '^kernel'                  # deve mostrar os 2 kernels
```

ROLLBACK: grubby --set-default-index=1 volta para o kernel anterior; --set-default-index=0 volta ao mais novo.

BLOCO 5 · Troubleshooting de permissões

Fluxo universal: ls -l → identificar problema → aplicar correção → testar.

Exemplo completo (Sim3·Q30): /home/secret_data está com permissões 000. Usuário sam (grupo admins) precisa ler: grupo dono = admins, owner=RW, group=R, others=nada.

```
ls -l /home/secret_data                            # ----- = 000, sem acesso algum

chown :admins /home/secret_data                     # muda so o GRUPO (":" antes do grupo)
chmod 640 /home/secret_data                         # 6=rw- owner | 4=r-- group | 0=--- others
ls -l /home/secret_data                            # esperado: -rw-r----- root admins

su - sam
cat /home/secret_data                              # deve funcionar
exit
```

Saída do ls -l	Significado	Correção
-----	Permissões 000 — sem acesso	chmod XXX + chown
-rw-r--r--+	ACL presente (sinal +)	getfacl / setfacl
drwxrws---	SGID ativo	normal em dirs colaborativos
lrwxrwxrwx	Link simbólico	ls -la para ver o alvo

Número	Binário	Permissão
7	111	rwX
6	110	rw-
5	101	r-X
4	100	r--
0	000	---

BLOCO 6 · Revisão rápida — os 10 fluxos mais críticos

Execute cada fluxo sem consultar nada. Meta: menos de 3 minutos por fluxo.

1 — Reset de senha root

```
# GRUB: 'e' -> na linha 'linux' trocar 'ro' por 'rw' -> adicionar no final: init=/bin/bash -> Ctrl+X
passwd root
touch /.autorelabel      # refazer labels SELinux no proximo boot
exec /sbin/init          # ou: /sbin/reboot -f
```

2 — LVM completo do zero

```
fdisk /dev/sdb  # n -> Enter -> Enter -> +2G -> w
partprobe /dev/sdb
pvcreate /dev/sdb1 && vgcreate myvg /dev/sdb1
lvcreate -n mylv -L 500M myvg
mkfs.ext4 /dev/myvg/mylv
mkdir /mylv
echo "/dev/myvg/mylv /mylv ext4 defaults 0 0" >> /etc/fstab
mount -a
lvextend -r -L +500M /dev/myvg/mylv  # -r tambem redimensiona o fs
```

3 — SELinux contexto + boolean

```
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
restorecon -Rv /web
setsebool -P httpd_use_nfs 1      # -P = persistente
getsebool httpd_use_nfs
```

4 — Systemd Timer completo

```
cat > /etc/systemd/system/job.service << 'EOF'
[Unit]
Description=Job
[Service]
Type=oneshot
ExecStart=/usr/local/bin/job.sh
EOF
cat > /etc/systemd/system/job.timer << 'EOF'
[Unit]
Description=Job Timer
[Timer]
OnCalendar=*-*-* 07:00:00
Unit=job.service
[Install]
WantedBy=timers.target
EOF
systemctl daemon-reload      # OBRIGATORIO antes de enable
systemctl enable --now job.timer
systemctl list-timers | grep job
```

5 — Diretório colaborativo SGID + ACL

```
groupadd devs && useradd -G devs alice && useradd bob
mkdir /collab && chown :devs /collab
chmod 2770 /collab           # 2=SGID
setfacl -m g:bob:rx /collab
setfacl -m d:g:bob:rx /collab # default (heranca)
getfacl /collab
```

6 — SSH sem senha + hardening

```
ssh-keygen -t rsa
ssh-copy-id user@host
ssh user@host      # TESTAR antes de desabilitar senha
vim /etc/ssh/sshd_config      # PasswordAuthentication no
sshd -T | grep -i passwordauthentication
systemctl restart sshd
```

7 — Repositório DNF local

```
vim /etc/yum.repos.d/local.repo
# [BaseOS]
# name=BaseOS
# baseurl=file:///mnt/BaseOS
# enabled=1
# gpgcheck=0
dnf clean all && dnf repolist
```

8 — ACL com negação de acesso

```
cp /etc/fstab /var/tmp/fstab_copy
setfacl -m u:alice:rw /var/tmp/fstab_copy
setfacl -m u:bob:--- /var/tmp/fstab_copy  # NEGAR bob (tres hifens)
getfacl /var/tmp/fstab_copy
```

9 — Firewall multi-camada (SELinux + firewall + serviço)

```
# Apache na porta 82 - tres camadas obrigatorias:
vim /etc/httpd/conf/httpd.conf          # Listen 82
semanage port -a -t http_port_t -p tcp 82
firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --reload
systemctl restart httpd
curl localhost:82
```

10 — Script shell com loop e condição

```
cat > /usr/local/bin/sum.sh << 'EOF'
#!/bin/bash
TOTAL=0
for NUM in "$@"; do
    if [ "$NUM" -gt 0 ]; then
        TOTAL=$((TOTAL + NUM))
    fi
done
echo "The sum is $TOTAL"
EOF
chmod +x /usr/local/bin/sum.sh
/usr/local/bin/sum.sh 10 -5 20 30      # The sum is 60
```

BLOCO 7 · Simulado mental + plano de estudos + dicas finais

Ideal: cubra a coluna "Resposta" e tente responder de cabeça primeiro. Errar mais de 3 = tema prioritário de revisão.

#	Pergunta	Resposta
1	Mudar SELinux para enforcing agora (sem reboot)?	setenforce 1
2	Arquivo para que enforcing persista após reboot?	/etc/selinux/config → SELINUX=enforcing
3	Criar VG 'myvg' com PE size 16MiB em /dev/sdb1?	vgcreate -s 16M myvg /dev/sdb1
4	Expandir LV e filesystem simultaneamente (+500M)?	lvextend -r -L +500M /dev/vg/lv
5	Adicionar IP secundário sem sobrescrever o primário (nmcli)?	nmcli con mod NOME +ipv4.addresses IP/MASK
6	Par de comandos para aplicar contexto SELinux a /web?	semanage fcontext -a -t TIPO "/web(/.*)?" + restorecon -Rv /web
7	Tornar um boolean SELinux persistente?	setsebool -P NOME 1
8	Ordem dos passos para ativar um systemd timer?	systemctl daemon-reload → systemctl enable --now NOME.timer
9	Formato do crontab para rodar às 12:45 AM todo dia?	45 0 * * * comando
10	Configurar umask 027 persistente para um usuário?	echo 'umask 027' >> /home/user/.bashrc
11	Criar diretório com SGID e grupo 'devs' correto?	groupadd devs && mkdir /dir && chown :devs /dir && chmod 2770 /dir
12	Negar acesso ao usuário 'bob' via ACL?	setfacl -m u:bob:--- /arquivo
13	Flag do tar para bzip2? E gzip?	-j = bzip2 -z = gzip -J = xz
14	Adicionar porta 82 ao tipo http_port_t no SELinux?	semanage port -a -t http_port_t -p tcp 82
15	Ativar swap sem reboot após adicionar no fstab?	swapon -a
16	Copiar chave SSH pública para servidor remoto?	ssh-copy-id user@servidor
17	Listar próximas execuções de todos os timers?	systemctl list-timers
18	Arquivo com política de expiração de senha para novos usuários?	/etc/login.defs → PASS_MAX_DAYS
19	Montar ISO em /mnt persistentemente no fstab?	/path/para.iso /mnt iso9660 loop,defaults 0 0
20	Tipo (filesystem) para ISO no fstab?	iso9660

Plano de estudos — reta final

Dia	Atividade	Objetivo
10/05	Simulados 3 e 4 cronometrados (2h30 cada)	Terminal limpo, sem consulta, anotar erros
11/05	Simulados 5 e 6 cronometrados (2h30 cada)	Mesmo esquema, foco em velocidade
12/05	Revisão dos erros dos dias 10 e 11	Refazer do zero só as que errou, sem consultar
13/05	Véspera — apenas cheatsheet	Nada novo. Dormir cedo
14/05	PROVA EX200	Ler cada tarefa com calma, verificar persistência

5 dicas finais para a prova

- ☐ **Persistência é tudo** — fstab, /etc/selinux/config, setsebool -P, systemctl enable, /etc/sysctl.d: se não sobrevive a um reboot, não conta ponto
- ☐ **Verifique o resultado** — curl localhost, df -h, getenforce, systemctl status, systemctl list-timers: nunca assuma, confirme
- ☐ **mount -a sem erro** — sempre após editar /etc/fstab, antes de continuar
- ☐ **daemon-reload antes do timer** — depois de criar/editar .service ou .timer
- ☐ **SELinux em dois passos** — setenforce (agora) + /etc/selinux/config (reboot); semanage fcontext + restorecon; setsebool -P

GUIA FINAL DE PROVA — EX200 RHEL 10

Dicas de prova remota · Gerenciamento de tempo · Verificação de persistência · Simulado mental completo com gabarito

Este guia é para usar nos dias 12 e 13 — véspera da prova. Não estude tema novo. Só revisar e executar.

BLOCO 1 · Como funciona a prova remota EX200

```
# FORMATO DA PROVA
Duração:      2 horas e 30 minutos
Pontuação:    0 a 300 pontos
Aprovação:    210 pontos (70%)
Tipo:         Performance-based (prática real, sem múltipla escolha)
Acesso docs:  Você TEM acesso a man pages dentro da VM do exame
Internet:     NÃO há acesso à internet externa
Verificação:  As correções são feitas APÓS reboot (persistência é obrigatória)

# AMBIENTE TÉCNICO (remoto)
- Você recebe uma VM RHEL 10 via browser (RHEL remote exam environment)
- O examinador vê sua tela via webcam
- Você precisa de: webcam, microfone, conexão estável (recomendado: cabo)
- Espaço de trabalho: mesa limpa, parede visível, sem segunda tela
- USB de boot: a Red Hat fornece – TESTAR com antecedência é crítico

# O QUE ESPERAR NO EXAME
- ~20 tarefas independentes
- Cada tarefa tem sub-tarefas (ex: "criar usuário E configurar sudo E testar")
- TUDO precisa persistir após reboot
- Cada objetivo vale pontos parciais – completar 70% de uma tarefa vale algo
- Não existe penalidade por tentativa – sempre tente

# REGRA DE OURO: VERIFICAR + REBOOT + VERIFICAR DE NOVO
Após cada tarefa importante, fazer a verificação. No final da prova:
REBOOT → confirmar que tudo ainda funciona.
```

BLOCO 2 · Gerenciamento de tempo na prova

Com 2h30 e ~20 tarefas, você tem aproximadamente 7 minutos por tarefa. Questões rápidas precisam sair em 2-3 minutos para sobrar tempo para as complexas.

```
# CLASSIFICAÇÃO DE TAREFAS POR VELOCIDADE

TAREFA RÁPIDA (2-3 minutos) – fazer primeiro:
- Reset de senha root          (fluxo mecânico, 2 min)
- Timezone + Chrony            (3 comandos, 2 min)
- Hostname                     (1 comando, 30 seg)
- Boot target padrão           (1 comando, 30 seg)
- Tuned profile                 (2 comandos, 1 min)
- Flatpak install              (2 comandos, 2 min)
- Sysctl forwarding            (1 arquivo + 1 comando, 2 min)
- Variável de ambiente global  (1 arquivo, 1 min)

TAREFA MÉDIA (4-6 minutos):
- Repositório DNF               (1 arquivo + verificação)
- Usuário com políticas         (3-4 comandos)
- SSH + hardening               (keygen + copy-id + config)
- Firewall                     (2-3 comandos)
- SELinux modo                  (2 comandos obrigatórios)
- Cron job                      (crontab -e)
- Systemd timer                 (2 arquivos + daemon-reload)
- ACL em arquivo                (2-3 setfacl)

TAREFA PESADA (7-12 minutos):
- LVM completo (fdisk→pvs→vgs→lvs→mkfs→fstab)
- LVM com PE size customizado
- VDO ou Thin Provisioning
- Rede nmcli com IPv4+IPv6+secundário
- Script shell complexo
```

- SELinux contexto (semanage+restorecon)
- Apache na porta não-padrão (3 camadas)

ESTRATÉGIA RECOMENDADA

1. Leia TODAS as tarefas primeiro (5 minutos)
2. Faça as RÁPIDAS primeiro – acumule pontos fáceis
3. Pule qualquer tarefa onde travar por mais de 2 minutos
4. Volte para as puladas com o tempo restante
5. Reserve 15 minutos no final para reboot e verificação final

SE TRAVAR EM UMA TAREFA:

- Use man para buscar a sintaxe exata
- Pule e marque para voltar
- NÃO passe mais de 12 minutos em uma única tarefa

BLOCO 3 · Checklist de persistência — o que DEVE sobreviver ao reboot

A verificação de persistência é o único critério real de correção do exame. Configurar e não persistir = zero pontos naquela tarefa.

PERSISTÊNCIA POR TIPO DE TAREFA

REDE (nmcli)

- connection.autoconnect yes no perfil
- Verificar: nmcli con show NOME | grep autoconnect

FSTAB (LVM, partição, NFS, swap)

- Entrada em /etc/fstab
- Verificar: mount -a (sem erro) + findmnt --verify

SERVIÇOS

- systemctl enable NOME (não só start)
- Verificar: systemctl is-enabled NOME

SELinux modo

- /etc/selinux/config SELINUX=enforcing
- Verificar: cat /etc/selinux/config

SELinux contexto de arquivo

- semanage fcontext -a -t TIPO "REGEX" (persiste)
- restorecon -Rv /dir (aplica no disco)
- Verificar: ls -Z /arquivo

SELinux boolean

- setsebool -P NOME 1 (o -P é a persistência)
- Verificar: getsebool NOME

Firewall

- firewall-cmd --permanent --add-service=... && --reload
- Verificar: firewall-cmd --list-all

Cron

- Salvo no crontab automaticamente ao sair do editor
- Verificar: crontab -l -u root

Systemd timer

- systemctl enable --now NOME.timer (não só start)
- Verificar: systemctl is-enabled NOME.timer

Usuário/grupo/senha

- Salvo automaticamente em /etc/passwd, /etc/shadow, /etc/group
- Verificar: id usuario, chage -l usuario

Hostname

- hostnamectl set-hostname (salva em /etc/hostname)
- Verificar: hostname, cat /etc/hostname

Sysctl

→ Arquivo em /etc/sysctl.d/99-arquivo.conf + sysctl -p
→ Verificar: sysctl net.ipv4.ip_forward

Umask

→ Linha no /home/user/.bashrc
→ Verificar: su - usuario && umask

Swap

→ UUID no /etc/fstab + swapon -a
→ Verificar: swapon -s, free -h

Boot target

→ systemctl set-default NOME.target
→ Verificar: systemctl get-default

Grubby

→ grubby --update-kernel=ALL (persiste automaticamente)
→ Verificar: grubby --info=DEFAULT | grep args

BLOCO 4 · Simulado mental final — escreva sem consultar

Para cada pergunta abaixo, escreva os comandos exatos ANTES de olhar o gabarito. Meta: responder tudo em menos de 45 minutos.

PERGUNTA 1: Como entrar no GRUB para reset de senha? O que trocar na linha linux?
PERGUNTA 2: Criar VG "dados" com PE 16M usando /dev/sdc1. LV "vol1" com 80 extents. Formatar xfs. Montar em /mnt/vol1 persistentemente.
PERGUNTA 3: Configurar enp0s3 com IP 10.0.1.5/24 GW 10.0.1.1 DNS 8.8.8.8 IPv6 fd02::5/64 GW fd02::1. IP secundário 172.16.0.5/24. Perfil: "corp".
PERGUNTA 4: Criar /web/html/index.html. Configurar SELinux para httpd servir /web. Confirmar persistência.
PERGUNTA 5: Configurar Apache na porta 8080. Todas as camadas necessárias.
PERGUNTA 6: Criar timer que roda /usr/local/bin/relatorio.sh todo dia às 08:30.
PERGUNTA 7: Script /usr/local/bin/check.sh que aceita nome de arquivo como argumento. Se existir: "Arquivo encontrado: NOME". Se não: "Arquivo ausente: NOME".
PERGUNTA 8: Criar grupo "analistas". Usuário "ana" (UID 2100) no grupo analistas. Diretório /dados/analistas com SGID. Outros sem acesso. ACL: usuário "bob" pode ler. Herança nos novos arquivos.
PERGUNTA 9: Configurar SSH para ana fazer login sem senha em localhost. Desabilitar autenticação por senha.
PERGUNTA 10: Configurar firewall: HTTP apenas para 10.0.1.0/24. Rejeitar de outros.

GABARITO DO SIMULADO MENTAL

GABARITO 1: GRUB → 'e' → linha 'linux' → trocar 'ro' por 'rw' → adicionar init=/bin/bash
Ctrl+X → passwd root → touch /.autorelabel → exec /sbin/init

GABARITO 2:

```
fdisk /dev/sdc # n → Enter → Enter → +2G → t → 8e → w
partprobe /dev/sdc
pvcreate /dev/sdc1
vgcreate -s 16M dados /dev/sdc1
lvcreate -l 80 -n vol1 dados
mkfs.xfs /dev/dados/vol1
mkdir /mnt/vol1
echo "/dev/dados/vol1 /mnt/vol1 xfs defaults 0 0" >> /etc/fstab
mount -a
df -h /mnt/vol1
```

GABARITO 3:

```
nmcli con add con-name corp ifname enp0s3 type ethernet ipv4.method manual ipv4.addresses 10.0.1.5/24 ipv4
nmcli con mod corp +ipv4.addresses 172.16.0.5/24
nmcli con up corp
ip addr show enp0s3
```

GABARITO 4:

```
mkdir -p /web/html
```

```
touch /web/html/index.html
semanage fcontext -a -t httpd_sys_content_t "/web(/.*)?"
restorecon -RV /web
ls -Z /web/html/index.html # confirmar httpd_sys_content_t
```

GABARITO 5:

```
vim /etc/httpd/conf/httpd.conf # Listen 8080
semanage port -a -t http_port_t -p tcp 8080
firewall-cmd --permanent --add-port=8080/tcp
firewall-cmd --reload
systemctl restart httpd
curl http://localhost:8080
```

GABARITO 6:

```
cat > /usr/local/bin/relatorio.sh << 'EOF'
#!/bin/bash
date >> /var/log/relatorio.log
EOF
chmod +x /usr/local/bin/relatorio.sh
cat > /etc/systemd/system/relatorio.service << 'EOF'
[Unit]
Description=Relatorio Diario
[Service]
Type=oneshot
ExecStart=/usr/local/bin/relatorio.sh
EOF
cat > /etc/systemd/system/relatorio.timer << 'EOF'
[Unit]
Description=Timer Relatorio 08:30
[Timer]
OnCalendar=*-*-* 08:30:00
Unit=relatorio.service
[Install]
WantedBy=timers.target
EOF
systemctl daemon-reload
systemctl enable --now relatorio.timer
```

GABARITO 7:

```
cat > /usr/local/bin/check.sh << 'EOF'
#!/bin/bash
if [ -f "$1" ]; then
    echo "Arquivo encontrado: $1"
else
    echo "Arquivo ausente: $1"
fi
EOF
chmod +x /usr/local/bin/check.sh
```

GABARITO 8:

```
groupadd analistas
useradd -u 2100 -G analistas ana
useradd bob
mkdir -p /dados/analistas
chown :analistas /dados/analistas
chmod 2770 /dados/analistas
setfacl -m u:bob:r /dados/analistas
setfacl -m d:u:bob:r /dados/analistas
getfacl /dados/analistas
```

GABARITO 9:

```
passwd ana # definir senha temporária
su - ana
ssh-keygen -t rsa # Enter em tudo
ssh-copy-id ana@localhost
exit
vim /etc/ssh/sshd_config # PasswordAuthentication no
```

```
sshd -T | grep passwordauthentication # verificar
systemctl restart sshd
ssh ana@localhost # confirmar acesso sem senha
```

GABARITO 10:

```
firewall-cmd --permanent --remove-service=http 2>/dev/null
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="10.0.1.0/24" service name="http
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" service name="http" reject'
firewall-cmd --reload
firewall-cmd --list-rich-rules
```

BLOCO 5 · Verificações finais antes do reboot na prova

```
# RODAR ANTES DO REBOOT FINAL:
systemctl list-timers # todos os timers ativos
systemctl --failed # serviços com falha
firewall-cmd --list-all # regras de firewall
getenforce # SELinux enforcing?
cat /etc/selinux/config | grep SELINUX= # persistente?
findmnt --verify # fstab tem erro?
swapon -s # swap ativa?
free -h # confirmar swap
ip addr show # IPs configurados?
nmcli con show | grep -i auto # autoconnect ativo?
tuned-adm active # perfil correto?
timedatectl # NTP sincronizado?
systemctl get-default # target padrão correto?
grubby --info=DEFAULT | grep args # parâmetros do kernel?
```

APÓS O REBOOT:

```
# Logar novamente e verificar os itens acima de novo
# Testar: curl localhost, ssh usuario@localhost, ls /mountpoint
# Qualquer item que falhou: você ainda tem tempo para corrigir
```

CINCO ERROS MAIS COMUNS QUE REPROVAM CANDIDATOS

1. Esquecer **systemctl ENABLE** (só deu start — não persiste no reboot)
2. Editar **/etc/fstab** sem rodar **mount -a** — descobrir o erro só depois do reboot travado
3. **lvextend** sem **-r** — LV expande mas filesystem fica do tamanho antigo
4. **semanage fcontext** sem **restorecon** — regra no banco, arquivo no disco continua errado
5. **setsebool** sem **-P** — boolean volta ao padrão no reboot